Titre Professionnel : Concepteur Développeur d'Applications **(CDA)**

Dossier de Projet personnel

Nom : Laurent AGOUROU

Années : 2024-2025

Organisme de formation : GRETA Poitou-Charentes

Projet : StockMeat Solutions - Application de gestion de stock de viande

1. Introduction ..... - 1 -
1.1 Contexte personnel ..... - 1 -
1.2 Objectif de l'application ..... - 1 -
2. Cahier des charges ..... - 2 -
2.1 Fonctionnalités principales ..... - 2 -
2.2 Fonctions contraintes ..... - 3 -
2.3 Technologies utilisées : ..... - 4 -
3. Tableau Kanban et User Stories ..... - 5 -
4. Modélisation de la BDD (Base de données) ..... - 7 -
4.1 Identification des entités métier ..... - 7 -
4.2 Définition des relations ..... - 8 -
4.3 Diagramme MPD ..... - 8 -
4.4 Tables MySQL ..... - 10 -
4.5 Modélisation des entités ..... - 13 -
5 Architecture de l'application ..... - 15 -
5.1 Couches ..... - 15 -
6 Fonctionnalités développées ..... - 17 -
6.1 Authentification ..... - 17 -
6.1.1 Authentification avec Spring Security ..... - 17 -
6.1.2 Réinitialisation du mot de passe par email ..... - 18 -
6.1.3 Déconnexion sécurisée de l'utilisateur ..... - 20 -
6.2 Gestion des utilisateurs ..... - 21 -
6.3 Gestion des produits ..... - 23 -
6.4 Scan et traçabilité ..... - 24 -
6.4.1 Scanner de code-barres physique ..... - 25 -
6.4.2 Code-barres ..... - 25 -
6.4.3 Configuration du scanner ..... - 26 -
6.5 Gestion du paiement ..... - 29 -
6.5.1 Fonctionnement global ..... - 29 -
6.5.2 Génération de la facture PDF ..... - 31 -
6.5.3 Envoi de la facture par mail ..... - 32 -
7 Déploiement automatisé avec Docker et GitHub Actions ..... - 33 -
7.1 Conteneurisation avec Docker ..... - 33 -
7.1.1 Dockerfile : Création de l'image Docker ..... - 33 -
7.1.2 Docker-compose : Coordination des services ..... - 34 -
7.1.3 Configuration du profil dev pour le lancement en Docker Compose ..... - 35 -

7.2 Automatisation du pipeline CI/CD avec GitHub Actions ..... - 38 -
8 Mise en ligne de l'application : VPS, SSH et nom de domaine ..... - 41 -
8.1 Configuration du serveur VPS pour héberger l'application ..... - 41 -
8.2 Sécurisation de l'accès distant via SSH ..... - 46 -
8.2.1 Sécurisation de l'accès avec SSL et proxy ..... - 50 -
9 Interface Utilisateur ..... - 52 -
10. Tests ..... - 54 -
10.1 Outils utilisés ..... - 54 -
10.1 Tests unitaires ..... - 55 -
10.2 Tests d'intégration ..... - 55 -
10.3 Rapports de couverture ..... - 56 -
11. Documentation ..... - 57 -
11.1 JavaDoc ..... - 57 -
11.2 Github et README ..... - 59 -
12 Conclusion ..... - 60 -
12.1 Rappel de l'objectif du projet ..... - 60 -
12.2. Synthèse des fonctionnalités réalisées ..... - 60 -
12.3 Méthodologie et bonnes pratiques ..... - 60 -
12.4 Ce que j'ai appris ..... - 61 -
12.5 Ouverture et perspectives ..... - 61 -
Annexes : ..... - 62 -

1. Introduction

1.1 Contexte personnel

L'année précédente, j'ai travaillé en tant qu'opérateur de nettoyage de nuit dans une entreprise de découpe de viandes, en qualité d'intérimaire. Chaque fin de semaine, avant le début du service, le chef d'équipe mettait à notre disposition des paquets de viande invendus ou issus du surplus. Nous pouvions alors choisir les produits parmi différents types de viandes : poulet, steak, côtes de porc, etc.

Une feuille était mise à disposition pour y inscrire notre nom, prénom, le type de viande choisi, ainsi que le nombre de paquets. À la fin du service, chaque employé devait régler en espèces auprès du chef de service le montant correspondant aux produits réservés.

Les prix étaient accessibles : – 1,00 € à 1,50 € pour les viandes non transformées (poulet, dinde, steak), simplement découpées – 2,50 € pour les viandes transformées (saucisses de Toulouse, chipolatas, merguez)

Ce système était entièrement manuel et physique, géré en interne par l'entreprise. Je me suis inspiré de cette expérience en tant qu'intérimaire pour concevoir mon application, que j'ai nommée **StockMeat Solutions**. Elle représente une solution adaptée pour lutter contre le gaspillage et les pertes alimentaires dans le secteur agroalimentaire, tout en rendant service aux employés.

1.2 Objectif de l'application

L'application **StockMeat Solutions** permet la gestion de produits alimentaires, principalement les viandes (transformés et non transformés) directement au sein de l'entreprise. Avec prise en charge :

- du scan de codes-barres de types EAN-13 (13 chiffres)
- du suivi des scans par utilisateur
- de la gestion des paiements avec génération et envoi de factures PDF.

Elle vise à offrir une interface simple et intuitive pour les employés de l'entreprise, qui pourront utiliser un scanner filaire (type Datalogic Touch TD1100) pour enregistrer un produit, et reposant sur un back-end sécurisé (Spring Boot + Spring Security)

2. Cahier des charges

2.1 Fonctionnalités principales

Gestion des utilisateurs :

- L'inscription : Création d'un nouveau compte si l'utilisateur n'en possède pas
- La Connexion : Authentification via l'email et mot de passe
- Mot de passe : Réinitialisation du mot de passe en cas d'oubli (email via Brevo)
- Profil : Possibilité de modifier son identifiant, mail ou mot de passe

Gestion des produits :

- Création : ajout d'un nouveau produit avec les champs requis (nom, code-barres, quantité, poids, prix, statut du stock et catégorie)

- Consultation : Affichage des produits existants et du stock disponible avec possibilité de filtrer par nom, catégorie ou code-barres ainsi que l'affichage du stock total

- Modification : Mise à jour des informations du produit

- Suppression : Suppression logique du produit

Scan des produits :

- Enregistrement : Création d'un nouvel enregistrement de scan par produit avec les champs requis (user_id, product_id, date_scan et payment_id)

- Consultation : Affichage du total de l'utilisateur

- Lien vers paiement : Possibilité de transformer un scan en paiement en reliant le produit scanné à un paiement

- Code-barres : Saisie du code-barres manuellement ou via un scanner filaire

Gestion des paiements :

- Enregistrement : Enregistrement des paiements dès le déclenchement du scan

- Type : Choix du type de paiement (CARD ou CASH)

- Consultation : Affichage du montant total des produits scannés

- Facture : Génération et envoi de facture PDF par mail

Interface web :

- Pages HTML/CSS/JSS : Création des pages web : Login, Envoi de lien au cas de mot de passe oublié, Réinitialisation de mot de passe, Profil, Produit, Scanner et Paiement

Sécurité :

- Authentification : Mise en place d'un système d'authentification sécurisé (Spring Security), avec gestion des sessions et des rôles utilisateur (ADMIN / USER).

- Mot de passe : Encodage des mots de passe utilisateur (BCrypt)

- Gestion des accès : Les pages publiques sont accessibles sans authentification

Les pages sensibles nécessitent une authentification préalable

Les ressources statiques sont librement accessibles.

2.2 Fonctions contraintes

Bases de données : Utilisation d'une base de données fiable permettant des sauvegardes régulières pour éviter des pertes de données

Déploiement : L'application doit être déployée dans des conteneurs pour qu'elle soit accessible dans différents environnements

Sécurité : Respects des bonnes pratiques de sécurité (HTTPS sur VPS, clé SSH pour déploiement, mot de passe hachés)

Test : Création de tests unitaires et d'intégration afin d'assurer la couverture du code et la vérification des principales fonctionnalités

Maintenabilité : Le code source doit être lisible, documenté (JavaDoc) et accompagné de bonnes pratiques de développement

Ergonomie : L'interface utilisateur doit rester simple, intuitive pour tous et utilisable depuis les principaux navigateurs

2.3 Technologies utilisées :

- **Backend** : Java 17, Spring Boot
- **Sécurité** : Spring Security (authentification + rôles)
- **Base de données** : MySQL
- **Frontend** : Thymeleaf, HTML, CSS, JavaScript
- **Tests** : JUnit 5, Mockito, MockMvc (unitaires & intégration)
- **Documentation** : JavaDoc, JaCoCo, Surefire (rapports HTML)
- **CI/CD** : GitHub Actions
- **Conteneurisation** : Docker & Docker Compose
- **Déploiement** : VPS (clé SSH + nom de domaine)
- **Mails** : Brevo (réinitialisation du mot de passe & envoi facture PDF)
- **PDF** : OpenPDF
- **Scanner code-barres** : Datalogic Touch TD1100

3. Tableau Kanban et User Stories

Ce tableau Kanban ci-dessous retrace les différentes tâches réalisées dans le cadre du développement de l'application StockMeat Solutions. Il illustre la progression des différentes fonctionnalités avec les colonnes : À faire, En cours et Terminé, permettant une visualisation claire de l'état d'avancement du projet. Chaque ligne représente une user story ou une tâche, associée à :

- Le nom de la tâche
- Une priorité (Très Haute ou Haute)
- Un statut (À faire, En cours et Terminé)
- Un rôle (Utilisateur ou Développeur)
- Une description de la tâche
- Un besoin exprimé sous forme d'user story (ex. : *En tant qu'utilisateur, je souhaite...*)

Ce tableau permet :

- De suivre l'évolution du projet en temps réel
- De répartir les responsabilités selon les rôles
- De prioriser les tâches critiques pour le bon fonctionnement de l'application
- De garantir que chaque fonctionnalité est développée selon un besoin clair

Tâches	Status	Rôle	Description	Critère d'acceptation	Priorité
Inscription utilisateur	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir m'in	Compte créé avec succès	Très Haute
Connexion/ Authentification	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir me	Session valide, pages protégées	Très Haute
Réinitialisation mot de passe	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite recevoir un e	Email envoyé via Brevo, token a	Haute
CRUD produit	Terminé	Développeur	En tant que développeur, je souhaite pouvoir	CRUD fonctionne et les validati	Haute
Gestion de stock	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite voir le statut	Changement du statut du stock	Très Haute
Scan produit	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite scanner un c	Code-barres EAN13 scanné via	Très Haute
Enregistrement paiement	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir enre	Paiement déclenché uniqueme	Très Haute
Génération facture PDF	Terminé	Utilisateur	En tant qu'utilisateur je souhaite recevoir par	PDF valide généré avec les proc	Haute
Conteneurisation de l'application	Terminé	Développeur	En tant que développeur, je souhaite mettre e	Un Dockerfile permet de créer	Haute
Déploiement sur VPS avec nom de domaine et accès SSH	Terminé	Développeur	En tant que développeur, je souhaite déployer	Le VPS est opérationnel et acce	Haute
Configuration de la base de données	Terminé	Développeur	En tant que développeur, je souhaite configur	La BDD est créée et accessibleL	Très Haute
Mise en place du CI/CD avec GitHub Actions	Terminé	Développeur	En tant que développeur, je souhaite automat	Un workflow `.github/workflow`	Haute
Implémentation des tests unitaires et d'intégration	Terminé	Développeur	En tant que développeur, je souhaite écrire de	Les tests unitaires couvrent les	Haute
Documentation du projet	Terminé	Développeur	En tant que développeur, je souhaite rédiger	Un fichier `README.md` est prése	Haute
Interface utilisateur	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir inte	Un site web avec plusieurs pag	Très Haute

Tableau Kanban

J'ai utilisé **Notion**, un outil de gestion tout-en-un qui permet de créer des tableaux interactifs pour organiser les tâches, suivre leur avancement, définir les rôles, les priorités et les critères d'acceptation. Grâce à son interface simple et flexible, Notion m'a permis de structurer efficacement le suivi de mon projet **StockMeat Solutions**.

Le tableau Kanban est disponible en intégralité en annexe afin de mieux de visualiser les user stories, la description des tâches, la priorisation de celle-ci et les critères d'acceptation.

(Annexe 1)

4. Modélisation de la BDD (Base de données)



4.1 Identification des entités métier

1ère étape de la modélisation de la BDD : On identifie les objets clés de l'application avec leur attribut :

- **User :** Représente un employé de l'entreprise, il peut effectuer des actions comme se connecter, scanner des produits, gérer ses paiements etc.

Rôle : Acteur principal du système et initiateur des interactions avec l'application.

Attributs : id, email, username, password, reset_token, token_expiration

- **Product :** Représente un produit stocké, scanné ou vendu. Il contient toutes les informations nécessaires à la gestion de stock et la traçabilité.

Rôle : Ressource centrale du stock, objet des opérations métier

Attributs : id, name, barcode, quantity, weight, price, status, category

- **Scan :** Représente une opération de lecture de code-barres par un utilisateur. Elle relie un produit à un utilisateur et à un futur paiement

Rôle : Représente une action faite par l'utilisateur, comme le scan d'un produit. Elle sert à relier l'utilisateur, le produit et le paiement

Attributs : id, user_id, product_id, date_scan, payment_id

- **Payment :** Représente une transaction financière liés à un ou plusieurs scans. Associé à un employé et contient les détails du paiement.

Rôle : Valide l'opération financière, preuve de transaction

Attributs : id, amount, type (ENUM CARD/CASH), payment_date, status, employee_id

4.2 Définition des relations

2ième étape : On définit les relations principales entre les entités

- Un utilisateur peut effectuer plusieurs scans (1-to-many)
- Un produit peut apparaître dans plusieurs scans (1-to-many)
- Un paiement peut être lié à plusieurs scans (1-to-many)
- Un paiement peut être créé par un utilisateur (User)

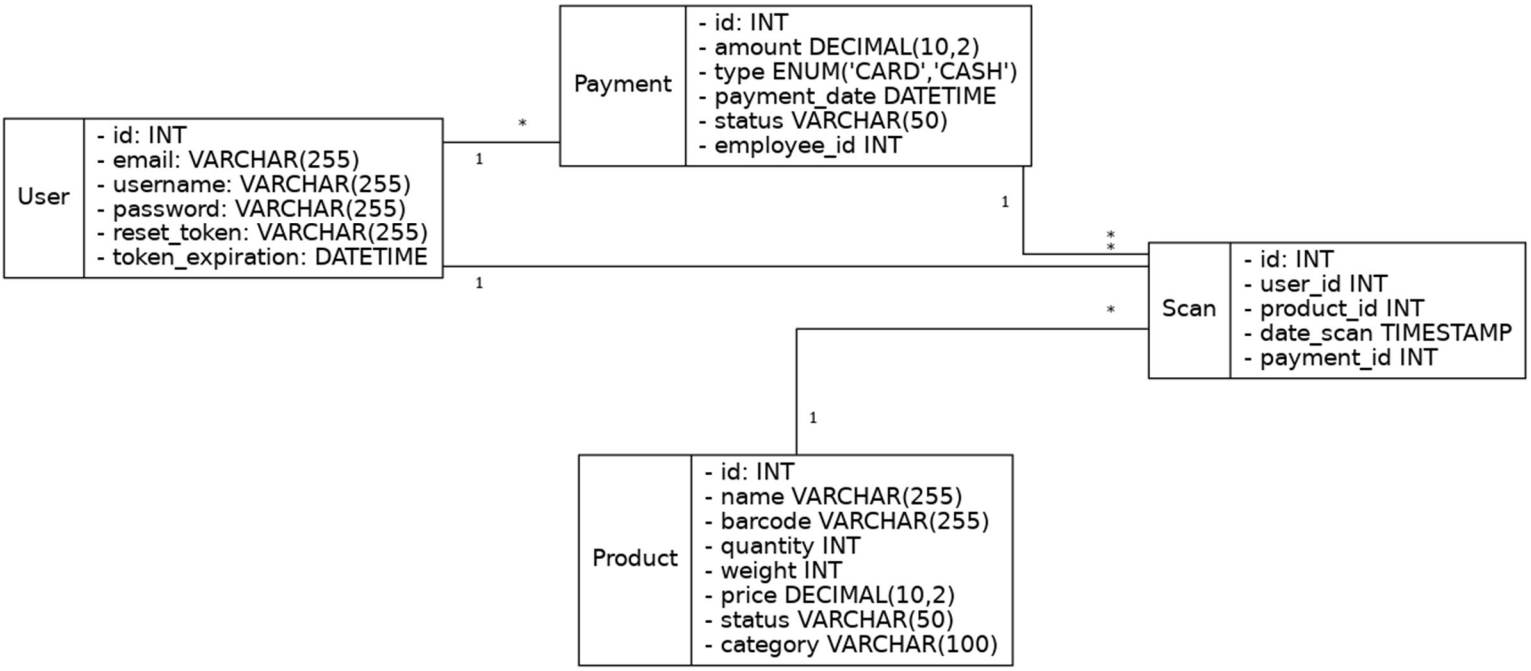


Diagramme de Classe : Visualisation des différentes entités et leur relation

4.3 Diagramme MPD

3ième étape : Création du diagramme MPD (Modèle physique de données) : Le MPD traduit les classes en tables SQL avec les types de données et les clés étrangères / primaires :

Les clés primaires servent à identifier chaque table de la BDD, elles sont uniques : **PK** = **id**

(PK : Primary Key)

Les clés étrangères servent à lier des tables entre elles :

(FK : Foreign Key)

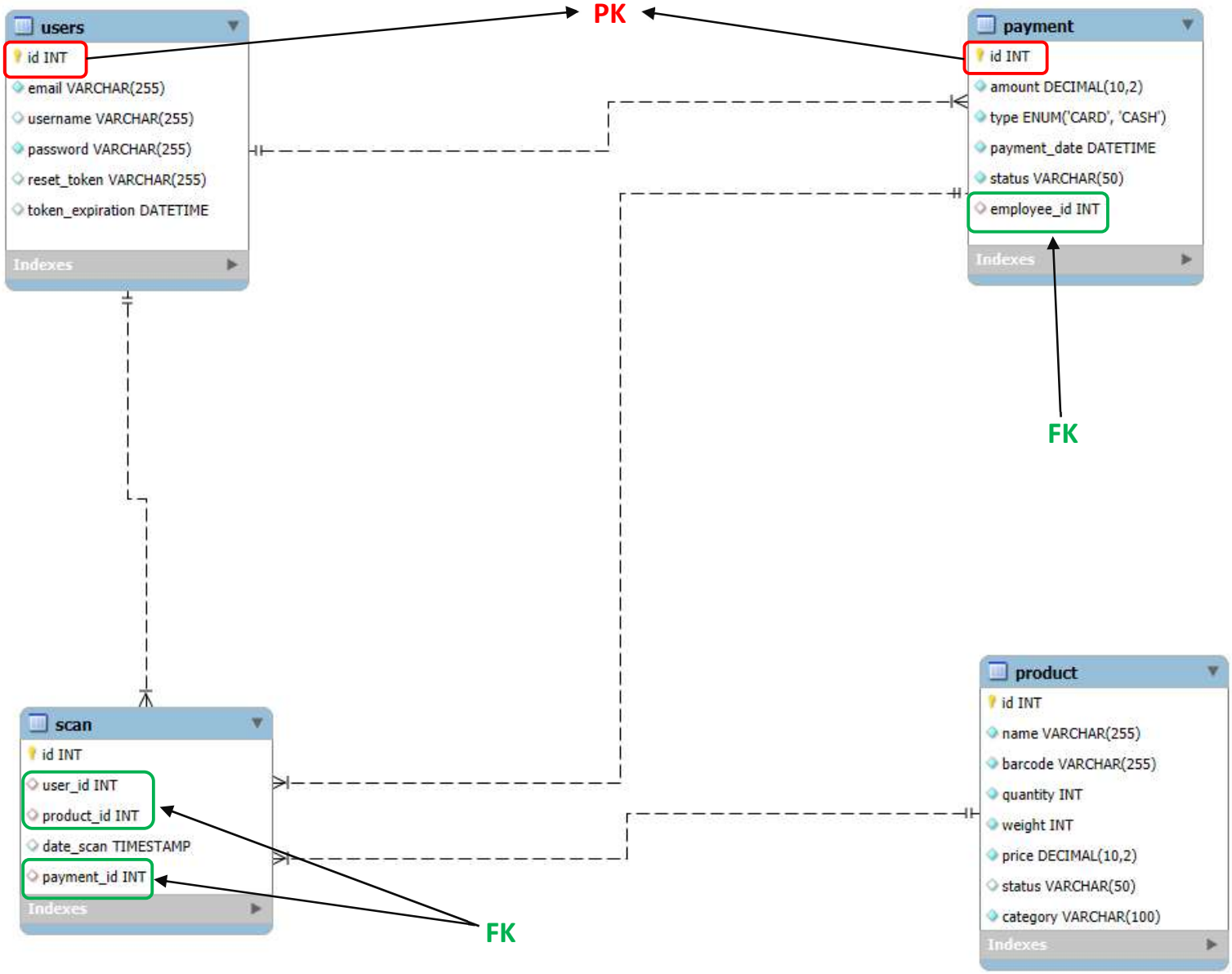
FK = **scan.user_id** fait référence à **user.id** afin de savoir qui a effectué le scan

FK = **scan. product_id** fait référence à **product.id** afin de savoir quel produit a été scanné

FK = **scan. payment_id** fait référence à **payment.id** afin de tracer les transactions liées à un ou plusieurs produits

FK = **payment. employee_id** fait référence à **user.id** afin de savoir quel employé a validé ou enregistré le paiement

Diagramme MPD



4.4 Tables MySQL

Dernière étape : Création des tables, pour ça j'ai utilisé **MySQL** un langage de base de données permettant justement la relation entre les différentes entités.

```sql
CREATE TABLE users (
  id INT NOT NULL AUTO_INCREMENT,
  email VARCHAR(255) NOT NULL UNIQUE,
  username VARCHAR(255),
  password VARCHAR(255) NOT NULL,
  reset_token VARCHAR(255),
  token_expiration DATETIME,
  PRIMARY KEY (id)
);
▷Run | ⧉Select
CREATE TABLE product (
  id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  barcode VARCHAR(255) NOT NULL UNIQUE,
  quantity INT NOT NULL,
  weight INT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  status VARCHAR(50),
  category VARCHAR(100) NOT NULL DEFAULT 'Autre',
  PRIMARY KEY (id)
);
```

```sql
CREATE TABLE scan (
  id INT NOT NULL AUTO_INCREMENT,
  user_id INT,
  product_id INT,
  date_scan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  payment_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES user(id),
  FOREIGN KEY (product_id) REFERENCES product(id),
  FOREIGN KEY (payment_id) REFERENCES payment(id)

);
▷Run | ⧉Select
CREATE TABLE payment (
  id INT NOT NULL AUTO_INCREMENT,
  amount DECIMAL(10,2) NOT NULL,
  type ENUM('CARD', 'CASH') NOT NULL,
  payment_date DATETIME NOT NULL,
  status VARCHAR(50) NOT NULL,
  employee_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (employee_id) REFERENCES employee(id)
);
```

Une capture d'écran présentée en annexe montre le contenu des tables **users** et **product** après insertion de données complètes. ***(Annexe 2)***

Pour connecter l'application à la base de données **MySQL**, j'ai configuré les paramètres dans le fichier **application. properties** qui sert à configurer l'application. Ces paramètres définissent l'URL de la base, le nom d'utilisateur, le mot de passe, et le **driver MySQL** utilisé par **Spring Boot**. J'ai aussi activé **JPA** pour gérer les entités et les relations entre les tables automatiquement.

Grâce à l'option **ddl-auto=update**, les tables sont créées ou mises à jour automatiquement à chaque redémarrage de l'application, en fonction des entités définies dans le projet.

Enfin, j'ai activé l'option **show-sql=true** pour voir les requêtes SQL dans la console, ce qui m'aide à suivre ce que fait l'application et à corriger plus facilement les erreurs pendant le développement.

À noter que l'application utilise plusieurs fichiers de configuration selon l'environnement dans lequel elle est lancée. Le fichier **principal application. properties** contient les réglages communs à tous les contextes. L'intégralité de ce fichier est présentée en annexe. ***(Annexe 3)***

Voici le fichier principal :

Application.properties :

```
# Jpa
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.jdbc.batch_size=20
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
```

Ensuite on a le fichier **application-dev. properties** qui permet à l'application de se lancer en **profil dev**, notamment via Docker **(Voir section *7.1 Conteneurisation avec Docker)***

Il configure une base MySQL nommée **mysqldb**, accessible dans le réseau **Docker** :

Application-dev. properties :

```
# Datasource
spring.datasource.url=jdbc:mysql://mysqldb:3306/project_apply?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
```

Après avoir mis en place le fichier principal et le fichier de configuration côté dev, j'ai choisi d'**activer** le **profil dev** dans le fichier central car c'est celui qui configure la base de données utilisée pendant les phases de développement, via le fichier **application-dev. properties**.

Cette structure permet d'isoler l'environnement dédié aux **développements**, avec une base dédiée, sans impacter la base de **production**.

Application.properties :

```
# Profil
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}
```

Pour le déploiement en production, on a également prévu un fichier de configuration spécifique **application-vps. properties** utilisé quand l'application est déployée sur un **serveur VPS**. Il utilise une base **MySQL locale** (localhost) avec des identifiants définis clairement.

Application-vps. properties :

```
# Datasource
spring.datasource.url=jdbc:mysql://localhost:3306/project_apply?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=rootroot
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
```

Enfin il est nécessaire d'isoler un environnement dédier aux **tests**, on utilise un fichier de configuration pour les tests, ce fichier est détaillé dans la section dédiée aux tests :

application-test. properties. ***(Voir section 10.2 Tests d'intégration)***

En résumé, cette organisation avec plusieurs profils permet à l'application de changer sa configuration selon l'environnement (développement, production, test) sans toucher au code.

4.5 Modélisation des entités

Une fois les tables créées dans MySQL, chaque entité est représentée dans le code par une **classe Java** appelée "**model**" (classes annotées avec **@Entity**), et grâce à **JPA**, ces entités sont automatiquement reliées à la base de données. Ce qui permet à l'application de **manipuler** les **données facilement.**

Ensuite, ces entités sont ensuite reliées à des interfaces repository, qui permettent d'accéder aux données en base sans écrire de requêtes SQL manuelles. Le fonctionnement précis de ces classes sera détaillé dans la section suivante sur l'architecture de l'application.

(Voir section 5 Architecture de l'application)

Pour mieux comprendre comment les entités Java sont reliées aux **tables MySQL**, voici un exemple concret avec la **classe User**. Cet exemple montre comment **JPA** utilise des annotations pour gérer les colonnes, les clés et les relations.

- La classe est annotée avec **@Entity** et **@Table (name = "users")**, ce qui indique à JPA qu'elle représente la **table users**.
    - Chaque champ Java correspond à une colonne SQL : par exemple, email, username, password, etc. Grâce à l'annotation **@Column**.
- L'annotation **@Id** indique la clé primaire, et **@GeneratedValue** permet l'**auto-incrément**.
- Les relations comme **@OneToMany** permettent de relier cette entité à d'autres, comme les Scan.

```sql
CREATE TABLE users (
    id INT NOT NULL AUTO_INCREMENT,
    email VARCHAR(255) NOT NULL UNIQUE,
    username VARCHAR(255),
    password VARCHAR(255) NOT NULL,
    reset_token VARCHAR(255),
    token_expiration DATETIME,
    PRIMARY KEY (id)
);
```

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "email")
    private String email;

    @Column(name = "username")
    private String username;

    @Column(name = "password")
    private String password;

    @Column(name = "reset_token")
    private String resetToken;

    @Column(name = "token_expiration")
    private LocalDateTime tokenExpiration;

    @OneToMany(mappedBy = "user")
    private List<Scan> scans;
```

Comme on peut le voir ci-dessus chaque champ de la classe Java correspond directement à une colonne de la table MySQL.

Ce principe est le même pour les autres entités comme Product, Scan ou Payment, ce dernier postulat clôt la modélisation de la base de données.

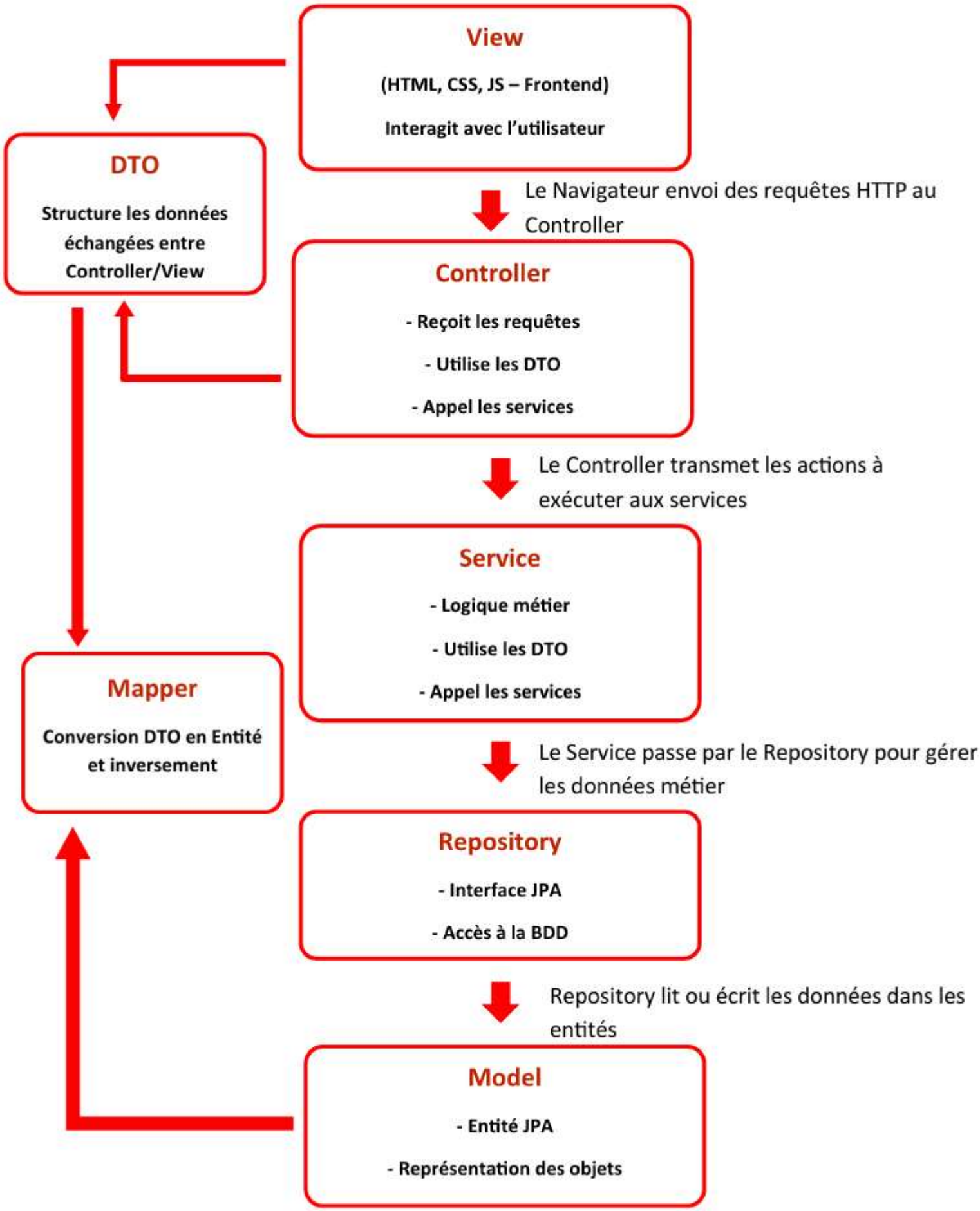
Les autres entités sont présentes en annexe ***(Annexe 4)***

5 Architecture de l'application

Afin d'organiser le code pour concevoir l'application, j'ai choisi l'architecture **MVC** (Model View Controller) c'est-à-dire qu'elle est organisée en plusieurs couches :

5.1 Couches

- **Model :** Entités JPA (User, Product, Scan, Payment) qui permet de stocker et récupérer les données des différents objets
- **Repository :** Interfaces utilisant Spring Data JPA ce qui permet d'interagir avec la BDD
- **Service :** Représente la logique métier, c'est-à-dire les règles et les actions que l'application doit suivre pour fonctionner correctement
- **Controller :** Représente la porte d'entrée de l'application, c'est-à-dire la partie qui reçoit les requêtes des utilisateurs et transmet les actions à exécuter aux services
- **DTO :** Représente un objet utilisé pour envoyer ou recevoir des données via l'API, sans exposer directement des données internes de l'application, ça permet de contrôler les informations partagées avec le navigateur web
- **Mapper :** Sert à convertir les Entités en DTO et inversement pour que les données soient bien préparées avant d'être envoyées ou reçues, la conversion est utile pour éviter d'envoyer trop d'informations sensibles ou inutiles au navigateur.
- **Configuration :** Regroupe les réglages techniques de l'application, notamment la sécurité avec Spring Security. Elle permet de définir qui peut accéder à quel page web, comment les utilisateurs sont authentifiés et comment les mots de passe sont protégés. Elle contient aussi les objets partagés (Beans) comme l'encodeur de mot de passe ou le gestionnaire d'authentification.
- **Exception :** Représente la gestion centralisée des erreurs dans l'application. Cela permet d'intercepter les problèmes (ex. : données invalides, accès refusé) et de renvoyer des messages clairs et contrôlés au navigateur.
- **View :** Représente l'interface utilisateur, composée de fichiers HTML, CSS et JavaScript. Elle permet d'afficher les pages (login, profil, produits, etc.) et d'interagir avec l'utilisateur.


View
(HTML, CSS, JS – Frontend)
Interagit avec l'utilisateur
DTO
Structure les données échangées entre Controller/View
Le Navigateur envoi des requêtes HTTP au Controller
Controller
- Reçoit les requêtes
- Utilise les DTO
- Appel les services
Le Controller transmet les actions à exécuter aux services
Service
- Logique métier
- Utilise les DTO
- Appel les services
Mapper
Conversion DTO en Entité et inversement
Le Service passe par le Repository pour gérer les données métier
Repository
- Interface JPA
- Accès à la BDD
Repository lit ou écrit les données dans les entités
Model
- Entité JPA
- Représentation des objets

6 Fonctionnalités développées

6.1 Authentification

6.1.1 Authentification avec Spring Security

Pour sécuriser l'accès à l'application, j'ai mis en place **Spring Security** dans la couche **Configuration.** Cela permet de gérer les connexions, les mots de passe, et les pages protégées.

Étapes réalisées :

- **SecurityConfig :** configure les règles d'accès en définissant les pages publiques qui sont **/login** pour l'authentification afin de se connecter.

J'utilise une authentification de session lorsqu'un utilisateur se connecte avec son email et son mot de passe, une session est créée automatiquement. Cette session permet de conserver l'état de l'utilisateur tout au long de sa navigation, sans avoir à se reconnecter à chaque page. Ensuite, l'utilisateur peut accéder à la page **/register** pour créer son compte en remplissant un formulaire d'inscription. Il peut aussi utiliser la page **/forget-password** pour réinitialiser son mot de passe en cas d'oubli ou de perte, afin de retrouver l'accès à son espace personnel.

Pour renforcer la sécurité, j'utilise **BCrypt** pour chiffrer les mots de passe avant de les enregistrer en base.

- **CustomUserDetailsService :** permet de retrouver un utilisateur en base à partir de son email, pour vérifier son identité lors de la connexion. Si l'utilisateur est trouvé, il est redirigé automatiquement vers son profil.
- **UserDetailsImplements :** transforme un utilisateur en format compatible avec Spring Security, sans exposer d'informations sensibles. Cela permet de sécuriser les données tout en assurant le bon fonctionnement du système d'authentification.

Le code source correspondant à cette fonctionnalité (fichiers SecurityConfig, CustomUserDetailsService, et UserDetailsImplements) est présenté en annexe. ***(Annexe 5)***

6.1.2 Réinitialisation du mot de passe par email

- **Configuration de l'envoi d'email avec Brevo** : Pour permettre à l'utilisateur de réinitialiser son mot de passe, j'ai configuré l'envoi d'email avec **Brevo**, un service en ligne français qui permet d'envoyer des emails via un **serveur SMTP** et une **clé API**. La configuration de **Brevo** se fait en ajoutant les paramètres dans le fichier **application. properties**, ces paramètres sont : le port sur lequel le serveur écoute, l'identifiant du serveur, son mot de passe et les options de sécurité nécessaires pour que l'envoi fonctionne correctement. Enfin, la **clé API** permet à l'application de s'identifier auprès de **Brevo.**

Une capture d'écran du mail reçu par l'utilisateur est visible en annexe. Ce message contient un lien sécurisé de réinitialisation, sur lequel l'utilisateur peut cliquer pour accéder directement à la page lui permettant de définir un nouveau mot de passe. ***(Annexe 6)***

Application.properties :

```
# Mail
spring.mail.host=smtp-relay.brevo.com
spring.mail.port=587
spring.mail.username=9583ad001@smtp-brevo.com
spring.mail.password=${SPRING_MAIL_PASSWORD}
spring.mail.protocol=smtp
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.debug=true
```

J'ai également développé un service **MailServiceImplements** qui utilise **JavaMailSender** pour envoyer les emails de réinitialisation. Lorsqu'un utilisateur demande à réinitialiser son mot de passe, un email est envoyé avec un lien sécurisé contenant un token valable 24h. Ce lien permet à l'utilisateur de choisir un nouveau mot de passe. Le code source correspondant à cette fonctionnalité les classes MailServiceIplements et JavaMailSender est en annexe. ***(Annexe 7)***

Une fois l'utilisateur connecté et authentifié, il peut accéder aux fonctionnalités internes de l'application, à commencer par la **gestion** complète **des utilisateurs**.

6.1.3 Déconnexion sécurisée de l'utilisateur

Pour permettre à l'utilisateur de quitter l'application en toute sécurité, j'ai mis en place une fonctionnalité de déconnexion via **Spring Security.**

Lorsqu'un utilisateur clique sur le bouton **« Se déconnecter »**, la session en cours est automatiquement fermée. Cela signifie que toutes les informations liées à son authentification sont supprimées, et qu'il ne peut plus accéder aux pages protégées sans se reconnecter.

La déconnexion est gérée par Spring Security avec l'URL **/logout**, qui est configurée dans la classe SecurityConfig. Une fois la session terminée, l'utilisateur est redirigé vers la page de connexion (/login) avec un message de confirmation.

Cette fonctionnalité est importante car elle permet de :

- Sécuriser les données en évitant qu'un autre utilisateur accède à l'espace personnel sans autorisation.
- Libérer les ressources et réinitialiser l'état de l'application
- Préparer une nouvelle connexion en revenant à l'accueil pour une future authentification

6.2 Gestion des utilisateurs 

En complément de l'authentification, j'ai mis en place une gestion complète des utilisateurs pour permettre à l'application de créer, modifier, consulter et supprimer des comptes de manière sécurisée et structurée.

J'ai développé cette partie en utilisant :

- Le service **UserService** centralise toutes les opérations liées aux utilisateurs :
- Création d'un **nouvel utilisateur** à partir **d'un formulaire d'inscription**, avec encodage du mot de passe via **BCrypt.**
- **Identification** d'un utilisateur à l'aide de son **ID** ou de son adresse **email**, avec message d'erreur si aucun résultat n'est trouvé.
- **Mise à jour** des informations personnelles (email, nom d'utilisateur, mot de passe) avec vérification des champs modifiés.
- **Suppression** d'un compte utilisateur.

Pour transporter les données entre frontend et backend, j'utilise le **DTO** (Data Transfer Object) appelé **UserDTO.** Ce principe est aussi utilisé pour d'autres types de données : **ProductDTO**, **PaymentDTO** et **ScanDTO**. Le code source de ces DTO est présenté en annexe. ***(Annexe 8)***

- Il contient les champs id, username, email et password, avec des contraintes de validation :

- **@NotBlank :** empêche les champs vides

- **@Size :** impose une longueur minimale et maximale

- **@Email :** vérifie le format de l'adresse email

Ces règles permettent de valider les données dès la saisie, avec des messages clairs en français affichés à l'utilisateur.

Pour la gestion des utilisateurs, on a trois **contrôleurs** distincts : **LoginController, RegisterController** et **ProfilController.**

- Le contrôleur **LoginController** gère **la connexion :**
- Gère l'accès à la page **/login**
- Créer une session sécurisée après que l'utilisateur s'est authentifié avec son email et son mot de passe.
- Redirige vers la page de connexion depuis la **racine /** c'est pour quand l'utilisateur **tape l'URL** du site dans le **navigateur web**, il est automatiquement redirigé vers la page de connexion (/login). Cela permet de démarrer l'application par l'accès sécurisé, sans tomber sur une page vide.

- Le contrôleur **RegisterController** gère **l'inscription :**
- Affiche le formulaire via la route **/register**
- Valide les données saisies avec les contraintes du **UserDTO** afin que l'utilisateur crée son compte correctement
- Vérifie que l'email n'est pas déjà utilisé si oui un message d'erreur s'affiche
- Encode le mot de passe et enregistre l'utilisateur
- Redirige vers la page de connexion avec un message de succès

- Le contrôleur **ProfilController** c'est la **modification de profil :**
- Permet à l'utilisateur connecté d'accéder à la page **/profil**
- Affiche ses données personnelles dans un formulaire prérempli
- Valide les modifications et les met à jour via **UserService**
- Synchronise **la session** après modification pour que les nouvelles données soient prises en compte immédiatement
- Affiche un message de confirmation en cas de succès

Le code source des classes utilisées pour la gestion des utilisateurs notamment UserService, LoginController, RegisterController et ProfilController est présenté en annexe. ***(Annexe 9)***

- L'**interface utilisateur** avec **Thymeleaf**, qui offre la possibilité à l'utilisateur d'interagir avec l'application en offrant une navigation simple et intuitive :
- Formulaire de connexion avec email et mot de passe
- Formulaire d'inscription avec validation des champs (identifiant, email et mot de passe)
- Messages d'erreur et de confirmation
- Lien vers la création de compte et la réinitialisation du mot de passe

Des captures d'écran sont disponibles en annexe pour illustrer les pages de connexion, d'inscription et de profil. ***(Annexe 10)***

Une fois la gestion des comptes utilisateurs en place, l'application peut se concentrer sur le suivi des **produits** et la **gestion du stock** en temps réel.

6.3 Gestion des produits 

Afin de permettre une gestion efficace des produits au niveau du système interne, il a fallu mettre en place le **CRUD** (Create, Read, Update, Delete) de cet **Endpoint**.

Pour cela, j'ai utilisé :

- Le modèle Product contient toutes les informations essentielles liées à un produit en stock :
  - nom, code-barres, poids, prix, quantité, statut et catégorie.
  - Des règles de vérification sont mises en place pour garantir que les données saisies sont correctes, comme par exemple un prix minimum de 0.10 € avec deux chiffres après la virgule.
  - Chaque produit peut aussi être **lié à des scans**, ce qui permet de **suivre les retraits effectués par les employés** et d'assurer une traçabilité complète du stock.

- Le service **ProductService** qui contient toute la logique métier liée aux produits :
  - Ajout ou mise à jour d'un produit selon son code-barres
  - Mise à jour automatique du **statut de stock** (rupture, faible, moyen, disponible)
  - Détection de la **catégorie** du produit selon son nom (ex. : bœuf → viande rouge)
  - Suppression d'un produit si le stock est disponible
  - Calcul du **stock total** pour l'affichage global

- Le contrôleur **ProductController** qui gère les routes **/products** :
  - Affichage de la liste des produits avec le stock total et le nombre de scans
  - Suppression d'un produit lorsque ce produit se fait scanner
  - Rafraîchissement manuel des statuts de stock

- L'**interface utilisateur** avec **Thymeleaf**, qui permet :
  - D'afficher les produits en stock avec leur nom, catégorie, poids, prix, quantité et statut
  - De visualiser le stock total et le nombre de produits scannés

Le code source des classes liées à la gestion des produits ProductService et ProductController est disponible en annexe. ***(Annexe 11)***

Pour mieux visualiser cette interface, une **capture d'écran** est disponible en annexe. Elle montre la page "Produit" avec le tableau du stock actuel, les statuts de chaque produit, ainsi que les indicateurs de stock total et de produits scannés. ***(Annexe 12)***

Une fois les produits enregistrés et organisés dans le stock, l'utilisateur peut les retrouver rapidement grâce à la fonctionnalité de **scan** par **code-barres**, qui simplifie la recherche et les opérations de suivi.

6.4 Scan et traçabilité 

La fonctionnalité de scan permet à l'employé de scanner le produit qu'il veut afin que celui-ci soit ensuite retiré du stock, cela permet à l'entreprise de conserver une certaine traçabilité des produits.

Pour cela, j'ai mis en place les éléments suivants :

- Le service **ScanService :**

- Enregistre chaque scan effectué par un utilisateur, avec la date et le produit concerné.
- Vérifie que le produit existe et que l'utilisateur est bien identifié.
- Permet de récupérer l'historique des scans par utilisateur.

- Le contrôleur **ScanController :**
- Gère-les routes **/scans** pour afficher la page de scan, enregistrer un scan ou réinitialiser le panier
- Récupère l'utilisateur connecté via **Spring Security.**
- Ajoute le produit scanné dans une **liste temporaire** affichée à l'écran, qui représente le **panier de l'utilisateur** pendant sa session.
- Cette liste est stockée en mémoire, et elle permet de visualiser les **articles scannés**, leur **prix**, et le **montant total** à payer. L'utilisateur peut ensuite choisir de **valider le paiement** ou de **vider le panier**, selon ses besoins.
- Calcule automatiquement le **montant total** des produits scannés.

- L'interface utilisateur avec **Thymeleaf** :
- Permet de **scanner** à l'aide d'un **scanner code-barres physique** ou de **saisir** un **code-barres** manuellement via un champ de saisie dédié.
- Affiche les **informations** du **produit scanné** : nom, catégorie, poids, prix.
- Affiche un **panier** avec les produits scannés et le total à payer.
- Propose deux actions :

  Passer au paiement pour finaliser l'opération.
  Vider le panier pour réinitialiser la session.

Le code source des classes liées à cette fonctionnalité ScanService et ScanController est en annexe. ***(Annexe 13)***

Une **capture d'écran** est disponible en annexe pour illustrer cette interface. Elle montre la page "Scanner" avec le champ de saisie, les articles scannés, le panier et les boutons d'action. ***(Annexe 14)***

6.4.1 Scanner de code-barres physique

Le scanner utilisé à été prêter par mon formateur, il est relié à l'ordinateur par un câble.

Cette douchette est un **Datalogic Touch TD1100**, un appareil léger et conçu pour une utilisation simple au quotidien. Il permet de lire les codes-barres par contact direct, en approchant le produit de la tête du scanner.

Ce modèle est particulièrement adapté aux environnements comme les boucheries, car il est **rapide**, **fiable** et **prêt à l'emploi sans configuration complexe**.



6.4.2 Code-barres

Pour identifier les produits de manière rapide et fiable, le système utilise des **codes-barres**. Il existe plusieurs types de codes-barres, mais dans le cadre de ce projet, j'ai choisi d'utiliser le format

EAN-13, composé de **13 chiffres**.

Ce type est **largement utilisé dans le secteur agroalimentaire**, notamment pour les produits emballés, car il permet une **standardisation internationale** et une **lecture facile par les scanners physiques**.

Un exemple de code-barres utilisé dans l'application est présenté en annexe. ***(Annexe 15)***

6.4.3 Configuration du scanner

Tout d'abord on branche le scanner à l'ordinateur via un **câble USB**, avec un adaptateur **USB** vers **port série (DB9),** la photo du branchement est disponible en annexe ***(Annexe 16).*** Cet adaptateur permet de relier ce type de matériel à un ordinateur moderne qui ont des ports USB.

Une fois branché le scanner est reconnu automatiquement comme un périphérique similaire à un clavier, il suffit ensuite de scanner un code-barres**,** et celui-ci est transmis directement dans le champ de saisie prévu à l'écran.

- Configuration logicielle : 

Pour que le scanner fonctionne automatiquement, j'ai utilisé un petit programme appelé **AutoHotKey**, c'est un outil gratuit pour **Windows** qui permet de créer des scripts (des mini programmes) : programme est disponible en annexe, sous le nom **scanner.ahk** ***(Annexe 17)***. Ce programme permet d'automatiser des actions, comme taper du texte ou cliquer à ta place.

Le script **AutoHotKey** sert à :

- Ouvrir le port série (**COM3**), là où le scanner est branché
- Régler la vitesse de communication des données qui est de **9600 bauds** soit un débit de **9600 bits par seconde**, ce qui permet aux scanner et l'ordinateur de se comprendre**.**
- Lire en contenu les données envoyées par le scanner.
- Nettoyer et convertir ces données en texte lisible
- Envoyer automatiquement le code scanné dans le champ saisi là où l'utilisateur à cliqué, cela se traduit comme si l'utilisateur l'avait tapé au clavier et ensuite appuyer sur Entrée.

Ce système permet au scanner de fonctionner comme un clavier, afin d'être compatible avec n'importe quelle zone de saisie.

- Configuration du scanner physique

Après avoir paramétré le scanner pour qu'il fonctionne comme un clavier, il est obligatoire de le configurer correctement en scannant certains codes de paramétrage fournis dans le guide du fabricant, ce guide est en annexe. ***(Annexe 18)***

Voici les étapes essentielles :

- **Scanner le code « Configuration par défaut »**



Cela permet de **réinitialiser le scanner** avec des réglages standards, pour éviter les conflits ou erreurs de configuration.

- **Scanner le code "RS232 Standard"**



Ce réglage active le **mode de communication série**, nécessaire pour que le scanner fonctionne avec le script **AutoHotkey** via le port de type **COM**. Le **mode série** est une façon d'envoyer les données **bit par bit**, dans un seul fil.

Ce réglage est nécessaire car il permet au scanner de parler le même langage que le script **AutoHotkey**, cela rend la **communication stable** pour l'ordinateur et c'est compatible avec les ports COM, utilisés dans le script.

- Fonctionnement côté interface web : 

Pour que le scanner fonctionne correctement avec l'application web, un petit script en **JavaScript** est utilisé dans la page **HTML.** Ce script vérifie si c'est l'utilisateur ou le scanner qui tape dans le champ prévu pour le code-barres. Il le vérifie en mesurant le temps entre chaque caractère saisi dans le champ.

Ce temps est appelé **delta**. Si ce delta est très court moins de **50 millisecondes**, cela signifie que les caractères ont été saisis très rapidement, comme le ferait un scanner. Dans ce cas, le script comprend que c'est un scan automatiquement et valide le formulaire tout seul, sans que l'utilisateur ait besoin de taper Entrée.

Sinon, le scan automatique n'est pas détecté, et cette fois-ci l'utilisateur entre manuellement le code-barres puis tape Entrée. Dans ce cas, le script vérifie si le code contient **13 chiffres** si oui le formulaire est envoyé sinon un message d'erreur s'affiche. Ce système permet de reconnaître les codes-barres **scannés automatiquement**, mais aussi de les **saisir manuellement** si besoin, au cas où le code-barres n'est pas lisible ou que le scanner ne le détecte pas.

Script (JavaScript)

```
<script>
  const input = document.getElementById("barcode");
  const scanForm = document.getElementById("scanForm");

  let lastInputTime = Date.now();

  input.addEventListener("input", function () {
    const now = Date.now();
    const delta = now - lastInputTime;
    lastInputTime = now;

    if (delta < 50 && input.value.length == 13) {
      scanForm.submit();
    }
  });

  input.addEventListener("keypress", function (e) {
    if (e.key === "Enter") {
      e.preventDefault();

      if (input.value.length === 13) {
        scanForm.submit();
      } else {
        alert("Le code-barres doit contenir exactement 13 chiffres.");
      }
    }
  });
</script>
```

6.5 Gestion du paiement

6.5.1 Fonctionnement global

Pour la partie paiement de mon application **StockMeat Solutions**, j'ai décidé de ne pas intégrer de service de paiement par carte (Visa, PayPal, MasterCard…) car ces services demandent des frais supplémentaires, et je préfère éviter des coûts en plus.

De plus, mon application s'inspire d'un système interne que j'ai connu en entreprise : chaque fin de semaine, les employés réglaient en espèces auprès du chef d'équipe. C'était un système simple, local et sans intermédiaire.

Dans cette optique, je n'ai pas voulu gérer de données bancaires, car ce sont des informations très sensibles. Cela permet de réduire les risques de fuites de données.

Le paiement se fait donc directement sur place, au sein de l'entreprise. Cela facilite la gestion du paiement par le chef de service, surtout que les horaires de travail sont parfois en dehors des heures d'ouverture classiques. Ce fonctionnement reste adapté au contexte réel de l'entreprise.

Ainsi, pour la fonctionnalité de paiement, j'ai choisi de partir sur un système de **réservation**. Cela signifie que lorsque l'employé souhaite finaliser son panier, il clique sur le bouton **"Passer au paiement"**, ce qui le redirige vers la **page de paiement**.

Pour mettre en place ce fonctionnement, j'ai développé les éléments suivants :

- Le service **PaymentService** : Gère toute la logique métier liée au paiement comme :

- Vérification du paiement en s'assurant que les produits ont bien été scannés avant de lancer le paiement.
- Calcul du montant total en regroupant les produits par code-barres, compte les quantités et calcule le prix total.
- Vérifie que le stock est bien disponible sinon le paiement ne se fait pas car le produit est en rupture de stock.
- Enregistrement du paiement en créant un objet Payment avec le montant, le type (espèces ou carte), la date et l'employé.
- Mise à jour du stock en retirant les quantités achetées du stock en base de données.
- Relie chaque scan au paiement effectué pour garder une traçabilité.
- Génération de la facture PDF : via le service **InvoiceService**, la facture est créée en mémoire.
- Envoi par email : la facture est envoyée automatiquement à l'adresse de l'employé via **MailService.**

Ce service est annoté avec **@Transactional**, ce qui garantit que toutes les opérations sont réalisées en une seule transaction : si une étape échoue, rien n'est enregistré. Elle sert aussi à rester cohérent avec la **BDD** en préservant l'intégrité des données, ce qui est indispensable dans une application de gestion comme **StockMeat Solutions**.

- Le contrôleur **PaymentController** : Régit les interactions entre l'utilisateur et le système :

- En gérant les routes : **/payment** qui affiche la page de paiement avec le récapitulatif des produits scannés et **/payment/confirm** qui déclenche le paiement lorsque l'utilisateur valide.
- En identifiant l'employé connecté grâce à **Spring Security**
- Réinitialisation du panier : une fois le paiement validé, le panier est vidé.

- L'interface utilisateur avec **Thymeleaf :** La page de paiement est conçue pour être simple et intuitive :

- Affichage du panier : liste des produits scannés avec leur nom et leur prix.
- Montant total : calculé automatiquement et affiché en bas de page.
- Choix du mode de paiement : l'utilisateur sélectionne entre **Carte** ou **Espèces**.
- Validation : un bouton permet de confirmer le paiement et de recevoir la facture.

Si aucun produit n'est présent dans le panier, le bouton est désactivé et un message s'affiche.

Ce système permet à l'employé de scanner ses produits, il passe ensuite au paiement via l'interface, choisit le mode de règlement par carte ou en espèces, et l'application web vérifie le stock, enregistre le paiement, met à jour les quantités, puis génère et envoie automatiquement une facture par email sous forme de PDF.

Une capture d'écran est disponible en annexe pour illustrer l'interface de paiement. Elle présente le récapitulatif des produits scannés, le montant total, le choix du mode de paiement, ainsi que le bouton permettant de valider l'achat et recevoir la facture. ***(Annexe 19)***

6.5.2 Génération de la facture PDF



Une fois le paiement validé, l'application génère automatiquement une facture au format PDF grâce au service **InvoiceService**. Ce document est créé en mémoire à l'aide de la bibliothèque **Open PDF**, sans passer par le système de fichiers.

La facture contient les éléments suivants :

- Le **logo de StockMeat Solutions**, centré en haut du document
- Un **titre clair** : "Facture – StockMeat Solutions"
- Le **numéro de facture**, généré dynamiquement selon le format : **F[année]- [identifiant du paiement sur 6 chiffres]** Par exemple : F2025-000017 signifie que la facture a été émise en 2025 et correspond au paiement n°17 dans la base de données. Ce format permet une **traçabilité simple** et **unique** pour chaque facture.
- Les **informations de l'employé** : adresse email
- La **date du paiement**, formatée en **jj/MM/aaaa**
- Le **mode de paiement** choisi (Carte ou Espèces)
- Un **message personnalisé** selon le type de paiement, pour rappeler à l'employé où et comment récupérer ses produits.

Ensuite, un **tableau récapitulatif** est généré automatiquement :

Produit	Quantité	Prix unitaire
Côte de porc	2	2.50 €

Ce tableau est construit à partir des données de scan liées au paiement. Le service **ScanRepository** permet de récupérer les produits scannés et leur quantité.

Enfin, le document affiche :

- Le **montant total à payer**
- Une **note de bas de page** : "StockMeat Solutions – Ne pas répondre à ce mail"

Le tout est encapsulé dans un **tableau PDF** bien mis en forme, puis converti en tableau de **bytes (byte [])** pour être envoyé par mail.

Une image de la facture générée est disponible en annexe. ***(Annexe 20)***

6.5.3 Envoi de la facture par mail



Une fois la facture PDF générée, elle est **envoyée automatiquement par email** à l'employé via le service **MailService**. Pour que cette fonctionnalité fonctionne correctement et que les mails soient bien reçus (sans aller en spam), j'ai configuré l'envoi à l'aide de la plateforme **Brevo.**

- **Configuration avec Brevo**

Voici les étapes techniques que j'ai réalisées :

- **Création d'un expéditeur vérifié** : J'ai enregistré l'adresse **noreply@stockmeatsolutions.site** comme expéditeur principal. Cette adresse est utilisée pour envoyer les factures aux employés.
- **Authentification du domaine** : Le domaine **stockmeatsolutions. site** a été **authentifié** directement dans l'interface **Brevo**. Cela permet de garantir que les mails envoyés sont bien légitimes et reconnus par les services comme **Gmail, Yahoo ou Outlook**.
- **Vérification de conformité** : Brevo confirme que le domaine respecte les **nouvelles exigences de sécurité** imposées par les principaux fournisseurs de messagerie (Google, Microsoft, Yahoo). Cela améliore la **bonne réception des mails** et évite qu'ils soient bloqués ou marqués comme indésirables.

- **Fonctionnement de l'envoi**

Le service MailService utilise **JavaMailSender** pour préparer et envoyer le message :

- Le mail contient :
    - L'**adresse mail** de l'**employé**
    - L'expéditeur : **noreply@stockmeatsolutions.site**
    - Le sujet : "Votre facture"
    - Le corps du message : "Veuillez trouver ci-joint votre facture."
    - La **facture PDF en pièce jointe**, nommée par exemple **facture_F2025-000017.pdf**
- Le message est envoyé automatiquement, sans intervention manuelle, dès que le paiement est validé.

Grâce à cette configuration avec Brevo, l'envoi des factures est **fiable et sécurisé**, ce qui garantit une bonne réception de la facture par les employés.

Une capture d'écran du mail reçu par l'utilisateur est en annexe. ***(Annexe 21)***

7 Déploiement automatisé avec Docker et GitHub Actions

Après avoir expliqué ce que l'application peut faire, je présente maintenant comment elle est installée et lancée facilement grâce à **Docker** et à l'automatisation avec **GitHub Actions**.

7.1 Conteneurisation avec Docker

Une fois que l'application fonctionne correctement, il est important de pouvoir la lancer facilement et rapidement, peu importe l'environnement (ordinateur local, serveur, cloud…). Pour cela, j'utilise **Docker**, qui permet d'empaqueter l'application avec tout ce qu'il faut pour qu'elle tourne partout de la même façon.

7.1.1 Dockerfile : Création de l'image Docker

Le Dockerfile est un fichier texte qui sert à construire une **image Docker** de l'application.

Cette image contient tout ce qu'il faut pour exécuter l'application dans un environnement isolé comme par exemple : le code, les dépendances, les outils, le configurations…, cela permet de ne pas dépendre du système local.

Dans mon projet, le **Dockerfile** permet de :

- Créer une image contenant l'application Java et ses dépendances.
- Faciliter le déploiement sur n'importe quel serveur ou machine compatible avec Docker
- Garantir que l'application fonctionne toujours de la même façon, peu importe l'environnement.

Voici le contenu du **Dockerfile** utilisé :

```
FROM openjdk:26-trixie

WORKDIR /app

COPY target/Project-Apply-0.0.1-SNAPSHOT.jar /app/stockmeatsolutions.jar

ENTRYPOINT ["java", "-jar", "/app/stockmeatsolutions.jar"]
```

- **FROM openjdk :26-trixie** : utilise une **image officielle Java** comme base, avec la **version 17**.
- **WORKDIR /app** : crée un dossier **/app** dans le conteneur et le définit comme dossier de travail.

- **COPY ...** : copie le fichier **.jar** généré par **Maven** dans ce dossier. Le fichier **.jar** (**J**ava **AR**chive) est le résultat du build **Maven** : il contient tout le code compilé de l'application, ainsi que ses dépendances. C'est ce fichier qui sera exécuté pour lancer l'application dans le conteneur.
- **ENTRYPOINT ...** : lance automatiquement l'application **Java** quand le conteneur démarre.

7.1.2 Docker-compose : Coordination des services

Après avoir créé une image Docker grâce au Dockerfile, il faut maintenant pouvoir **lancer l'application** avec ses **dépendances**, comme la base de données MySQL. C'est là que **Docker Compose** entre en jeu.

Docker Compose est un outil qui permet de définir et de démarrer plusieurs conteneurs en une seule commande, à partir d'un fichier ***docker-compose.yml***. Il facilite l'organisation des services, en précisant comment ils doivent communiquer, démarrer, et partager des ressources.

Dans mon projet, Docker Compose permet de :

- Connecter l'**application Spring Boot** à une **base de données MySQL.**
- Configurer les variables d'environnements nécessaires au bon fonctionnement (base de données, profil Spring, configuration mail…), grâce au fichier**. env** qui centralise ces paramètres. Ce fichier permet de **séparer les données sensibles et les réglages spécifiques** du code source, ce qui rend ton application plus sécurisée et plus facile à adapter selon l'environnement. Par exemple, dans le fichier**. env**, on peut définir : **SPRING_DATASOURCE_PASSWORD=rootroot**. Cela permet de **garder le mot de passe de la base de données à part du code**.

Le fichier**. env** est disponible en annexe mais les mots de passe et identifiants sensibles ont été masqués pour des raisons de sécurité. ***(Annexe 22)***

- Gérer les **volumes**, c'est-à-dire des espaces de stockage qui permettent de garder les données même si le conteneur est arrêté ou recréé. Par exemple, les données de la base **MySQL** sont enregistrées dans un volume nommé **mysql-data**, ce qui évite de tout perdre à chaque redémarrage.
- Créer un réseau privé pour que les services puissent communiquer entre eux en toute sécurité. Par exemple, le réseau **spring-boot-mysql-network** permet à **mysqldb** et **stockmeatsolutions** de **se reconnaître** et **échanger des données** sans conflit.

Le fichier **docker-compose.yml** contient plusieurs blocs importants :

- **services** : définit les deux conteneurs à lancer :
  - **mysqldb** : conteneur **MySQL** avec sa configuration.
  - **stockmeatsolutions** : conteneur de l'application **Spring Boot**, qui **dépend de MySQL**.

- **depends_on** : indique que l'application doit attendre que la base de données soit prête avant de démarrer.
- **healthcheck** : vérifie que MySQL est bien lancé et opérationnel avant de démarrer l'application.
- **volumes** et **networks** : comme expliqué précédemment, ils servent à conserver les données et d'assurer la communication entre les services.

Les fichiers **Dockerfile** et **docker-compose.yml** sont situés à la racine du projet, ce qui facilite leur détection et leur exécution automatique par les outils comme **Docker** et **GitHub Actions**.

Le fichier complet **docker-compose.yml** est disponible en annexe pour plus de détails. ***(Annexe 23)***

Afin de gérer les conteneurs au quotidien, j'utilise des commandes simples de docker, les commandes sont listées en annexe, avec leur rôle et syntaxe. ***(Annexe 24)***

7.1.3 Configuration du profil dev pour le lancement en Docker Compose

Pour que l'application fonctionne correctement dans son conteneur **Docker**, le fichier **docker-compose.yml** doit être activé par le **profil dev.**

Afin de faire cela, on injecte une variable d'environnement à l'intérieur du fichier **docker-compose.yml** :

Docker-compose. yml :

```
environment:
  - SPRING_PROFILES_ACTIVE=dev
```

Ce profil permet de charger une configuration spéciale définie dans le fichier **application-dev. properties**, adaptée à l'environnement de développement : connexion à la base ***MySQL (Voir section 4.4 Tables MySQL)***

et activation des outils techniques comme **health, info ou env,** via le bloc **Actuator**.

Ces éléments permettent de vérifier si **l'application est bien lancée**, **connectée** et **prête**, ce qui est très utile en **environnement Docker** pour le suivi technique.

Application-dev. properties

```
# Datasource
spring.datasource.url=jdbc:mysql://mysqldb:3306/project_apply?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver

# Actuator
management.endpoints.web.exposure.include=env,health,info,mappings
```

Pour vérifier facilement que l'application fonctionne bien en mode développement dans son conteneur Docker, j'ai ajouté un contrôleur simple appelé **StatusController**.

Ce contrôleur est actif uniquement avec le **profil dev** et expose **l'URL /api/status**, qui renvoie une **réponse JSON** indiquant que l'application est lancée, que le profil est correct, et que tout fonctionne.

C'est un point de contrôle rapide, utile pendant les phases de test ou de configuration.

StatusController

```java
@Profile("dev")
@RestController
@RequestMapping("/api")
@Slf4j
public class StatusController {

    @GetMapping("/status")
    public Map<String, String> status() {
        log.info("Endpoint appelé /api/status appelé");
        return Map.of(
                k1:"status", v1:"ok",
                k2:"environment",
                v2:"dev",
                k3:"message", v3:"Application is running");
    }

    @PostConstruct
    public void init() {
        log.info("StatusController chargé avec le profil DEV");
    }

}
```

Grâce à **Docker** et **Docker Compose**, l'installation et le démarrage de l'application sont faciles, rapides et ne dépendent plus du système en local. Tout est automatisé pour que l'application fonctionne de la même façon partout.

Pour aller plus loin dans l'automatisation, j'ai mis en place un système **CI/CD** avec **GitHub Actions**

7.2 Automatisation du pipeline CI/CD avec GitHub Actions 

Une fois l'application conteneurisée avec **Docker**, il est utile d'automatiser les étapes de test, de construction et de déploiement. C'est exactement le rôle d'une **pipeline CI/CD**

- Qu'est-ce qu'une pipeline CI/CD ?

La pipeline CI/CD (Intégration Continue / Déploiement Continu) est un **ensemble d'étapes automatisées** qui s'exécutent à chaque modification du code. Elle permet de :

- Vérifier que le code fonctionne ***(tests unitaires et d'intégration, voir partie 8.1)*** : cette étape permet de s'assurer que chaque partie du code fait bien ce qu'elle est censée faire, et que l'ensemble de l'application réagit correctement dans des situations réelles. Cela évite de déployer du code qui pourrait provoquer des erreurs ou des comportements inattendus une fois en production.
- Construire l'application, c'est-à-dire générer le **fichier .jar** spontanément
- Déployer automatiquement sur le serveur ***(VPS, voir partie 8.2)*** si tout est validé

Cela évite les erreurs manuelles, accélère les mises à jour, et garantit que l'application reste stable et fonctionnelle à chaque modification. En automatisant ces étapes, on s'assure que chaque nouvelle version du code est testée, construite et déployée de manière fiable et cohérente.

- Le fichier **ci-cd.yml**

Ce fichier est situé dans le dossier**. github/workflows/**, lui-même placé à la racine du projet. Cette structure est obligatoire pour que **GitHub** détecte automatiquement le **workflow** et puisse l'exécuter à chaque modification du code (push ou pull request sur la branche main).

Le fichier **ci-cd.yml** décrit les **étapes** de la **pipeline**. Voici ses principales fonctionnalités :

- **Déclenchement automatique** : la pipeline démarre à chaque push ou pull request sur la branche main, Cela signifie que dès qu'un développeur modifie le code, il envoie ses changements sur GitHub avec un **commit** (un petit message qui décrit la modification). Une fois ce push effectué, GitHub détecte l'action et **lance automatiquement la pipeline.**

- **Installation de l'environnement** : la pipeline prépare automatiquement une machine virtuelle sous **Ubuntu**, installe **Java 17** (nécessaire pour exécuter l'application) et configure **Maven** (l'outil utilisé pour compiler le projet et gérer les dépendances). Cela garantit que le code est toujours testé et construit dans un environnement propre et cohérent.

- **Vérification du code** : exécution de tous les tests (unitaires et intégrations) avec la commande Maven **mvn verify**

- **Transfert des fichiers** : cette étape copie automatiquement les fichiers nécessaires au déploiement le **.jar**, le **Dockerfile** et le **docker-compose.yml** vers le serveur distant. Le transfert se fait via **SSH** (un protocole sécurisé qui permet de communiquer avec un serveur à distance comme si on était connecté dessus), ce qui garantit que les fichiers arrivent en toute sécurité, sans intervention manuelle.

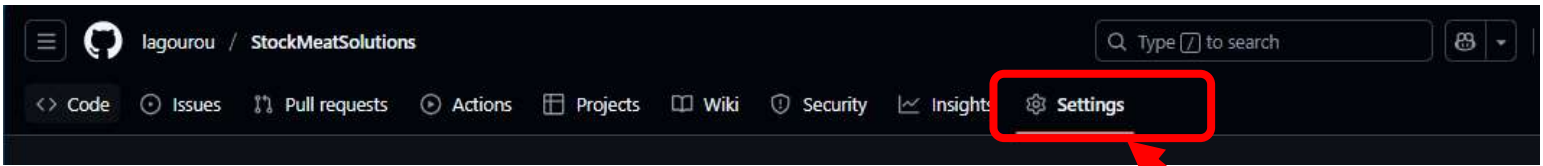
- **Déploiement sur le VPS** : une fois les fichiers transférés, le serveur commence par **arrêter les anciens conteneurs** pour éviter les conflits. Ensuite, il **reconstruit les images Docker** à partir des fichiers reçus, puis **relance les services** de l'application. Une fois le tout en place, il **nettoie les images inutiles** pour libérer de l'espace. Enfin, une **requête HTTP est envoyée** pour vérifier que l'application fonctionne correctement et répond comme prévu.

Le fichier complet **ci-cd.yml**, avec toutes les étapes décrites ci-dessus dans l'ordre d'exécution réel (du déclenchement jusqu'au déploiement), est disponible en annexe. ***(Annexe 18)***

Pour que le fichier **ci-cd.yml** fonctionne correctement tout en restant sécurisé, certaines informations sensibles ne sont pas écrites directement dans le code. Elles sont gérées séparément dans la configuration du dépôt **GitHub.**

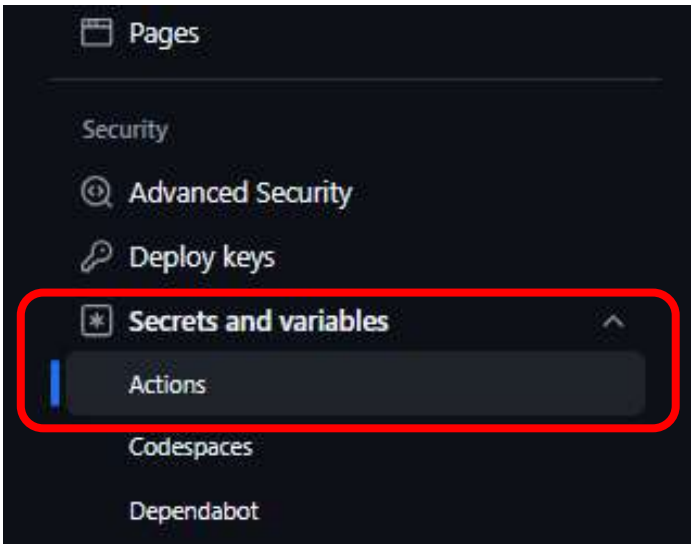
Ces informations sensibles (mot de passe, clé SSH, IP du serveur…) sont stockées dans la section **"Secrets and variables"** du dépôt GitHub :

- Je vais dans le **repo Github** du projet ensuite je rentre dans les paramètres :

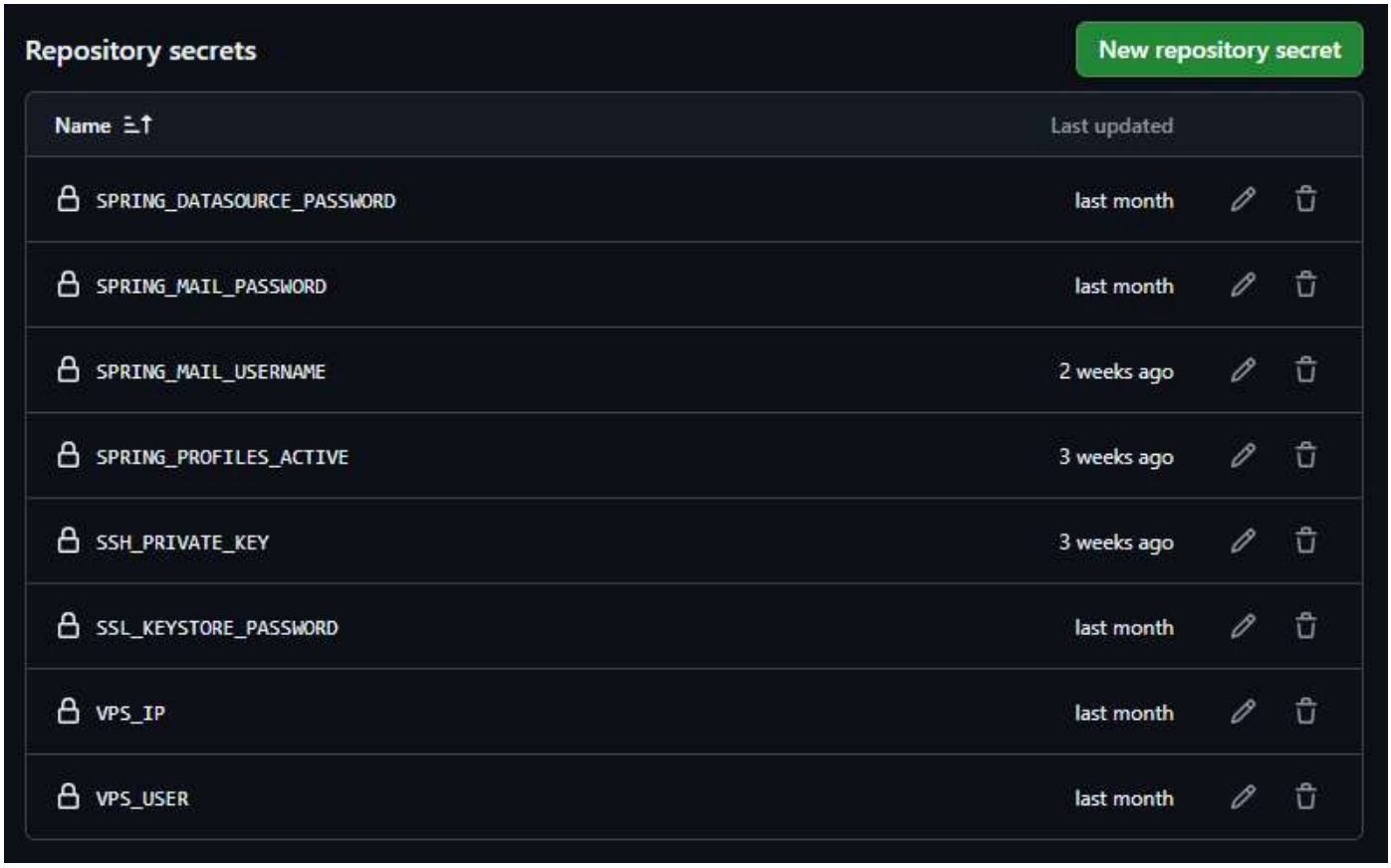


Paramètres

- Pour ensuite aller dans **"Secrets and variables"** et dans **Actions** de **Github actions**



- Enfin on peut ajouter les informations sensibles dans la section dédiée du dépôt GitHub. Cela permet de sécuriser des données comme les mots de passe, les clés SSH ou l'adresse IP du serveur, sans les écrire directement dans le code. Ces secrets sont utilisés automatiquement dans la pipeline CI/CD grâce à la syntaxe **${{secrets.NOM_DU_SECRET}}** dans le fichier **ci-cd.yml**, et également dans le fichier **docker-compose.yml** via des variables comme **${SPRING_MAIL_PASSWORD}.** Cela garantit que les données confidentielles restent protégées tout en étant accessibles au moment du déploiement.



En utilisant **Docker** et **GitHub Actions** ensemble, le projet peut être installé automatiquement et de façon sécurisée. L'application est testée, construite et lancée facilement sur n'importe quel serveur, sans avoir besoin de configurer chaque machine en local. Cela permet d'éviter les erreurs, de gagner du temps, et de garder le code fiable tout en protégeant les informations sensibles.

Une fois l'application prête à être déployée automatiquement, il faut maintenant la rendre accessible depuis Internet. Cela passe par la configuration du **serveur** (**VPS**), la **connexion sécurisée** (**SSH**), et l'association d'un **nom de domaine**.

8 Mise en ligne de l'application : VPS, SSH et nom de domaine

Pour héberger l'application en ligne, il faut d'abord disposer d'un serveur accessible à distance. C'est le rôle du VPS (Virtual Private Server), un serveur privé virtuel qui permet d'installer, configurer et faire tourner l'application comme si elle était sur une vraie machine physique.

8.1 Configuration du serveur VPS pour héberger l'application

Pour héberger l'application en ligne, j'ai choisi un **VPS (Virtual Private Server)** chez **LWS**, un hébergeur français reconnu pour ses offres accessibles, ses serveurs localisés en France, et son interface de gestion claire **(LWS Panel)**.

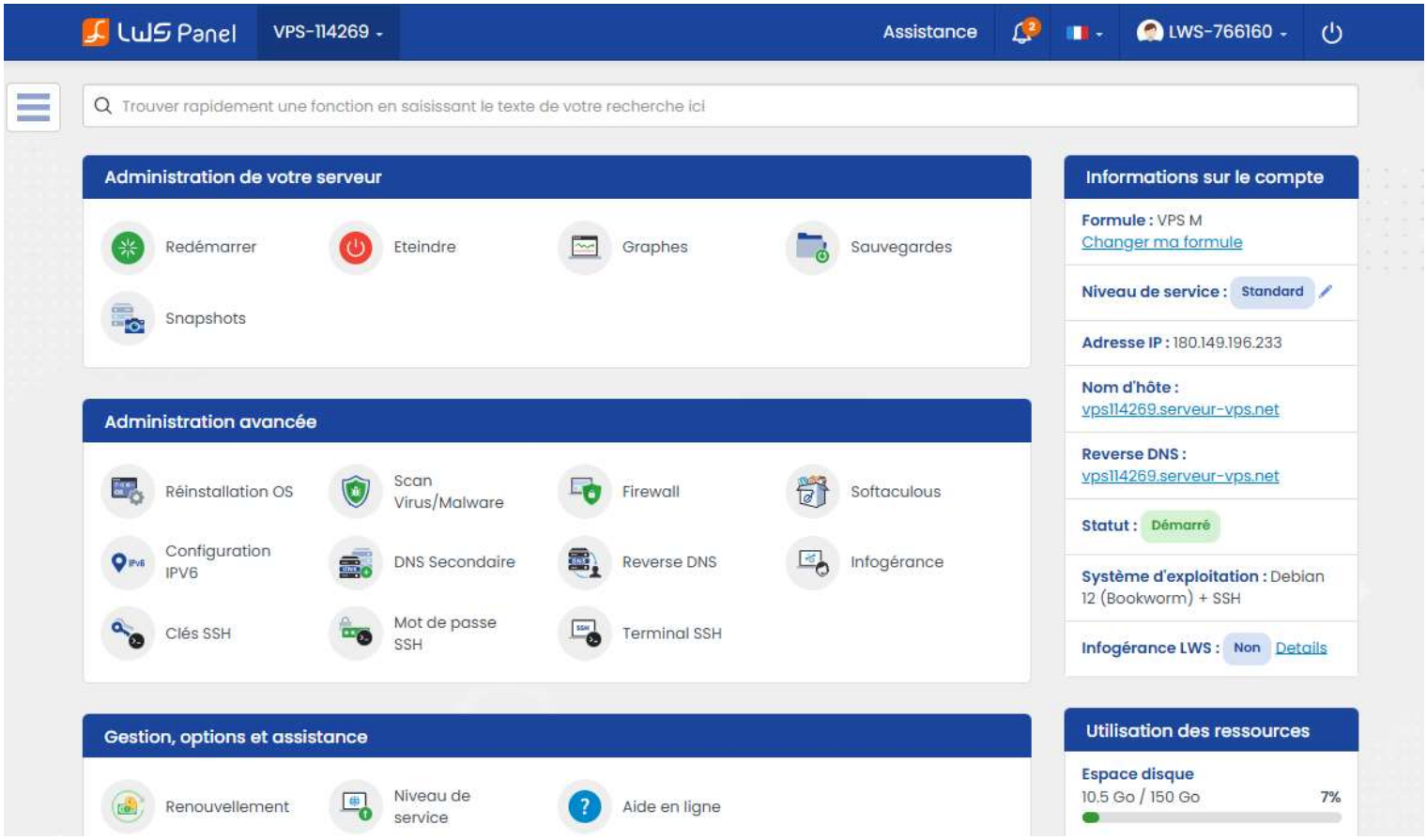
Le **LWS Panel** offre une interface claire et intuitive qui permet de gérer facilement les principaux paramètres du serveur.

Grâce à cela, il est possible de suivre l'état du VPS, d'intervenir en cas de besoin (comme redémarrer le système), de contrôler les règles de sécurité réseau via le pare-feu, et d'accéder à la console SSH pour effectuer des opérations techniques directement sur le serveur.

Cette interface centralise les outils essentiels pour assurer le bon fonctionnement et la maintenance du VPS au quotidien.

Mais plutôt que d'utiliser l'interface du **LWS Panel** pour effectuer les premières configurations, j'ai préféré passer directement par le **terminal Ubuntu**

Interface du panel LWS



- Choix du système d'exploitation :

Ensuite il fallait choisir le système d'exploitation du coup, j'ai choisi **Debian 12 (Bookworm)** car c'est un système stable et léger pour les serveurs. Il est très proche d'**Ubuntu**, que j'utilise en local, donc je retrouve les mêmes commandes et outils, ce qui facilite la configuration du VPS à distance via SSH.

Elle bénéficie de mises à jour régulières, et d'une large compatibilité avec les outils comme **Java, Docker, Maven** ou encore **GitHub Actions**.

Debian est aussi réputée pour sa **fiabilité en production**, ce qui est essentiel pour héberger une application métier comme **StockMeat Solutions**.

- Étapes de configuration du VPS :

Ces étapes sont **indispensables** pour sécuriser et préparer correctement le serveur. Elles permettent de partir sur une base saine, à jour et fonctionnelle, quel que soit l'environnement.

En configurant le système dès le départ, on évite les erreurs, les failles de sécurité, et les conflits lors de l'installation des outils comme Docker ou Java.

Une fois le **VPS** activé, voici les premières commandes à exécuter dans le terminal pour sécuriser et préparer le serveur :

1. Mettre à jour les paquets du système :

Cette commande permet de **mettre à jour le système du VPS**. Elle télécharge la liste des dernières versions disponibles **(apt update)** et installe automatiquement les mises à jour **(apt upgrade -y).** C'est une étape essentielle pour avoir un serveur sécurisé, stable et prêt à accueillir les outils nécessaires

```
sudo apt update && sudo apt upgrade -y
```

2. Créer un nouvel utilisateur non-root

Le **compte root** est l'utilisateur principal du système Linux, avec tous les droits d'administration. Il est créé automatiquement lors de l'installation du VPS et permet de tout gérer, mais son usage direct peut être risqué.

Pour renforcer la sécurité, on crée un **utilisateur non-root** (ex. **stockmeatsolutions**), ce qui permet d'administrer le serveur sans exposer le compte **root**.

```
sudo adduser stockmeatsolutions
sudo usermod -aG sudo stockmeatsolutions
```

Après avoir ajouté un nouvel utilisateur il est important de définir un mot de passe suffisamment sécurisé.

Ce mot de passe servira à te connecter avec cet utilisateur via SSH.

3. Activer le pare-feu UFW

Ces commandes permettent d'activer le pare-feu UFW sur le VPS. On autorise les connexions SSH pour pouvoir continuer à administrer le serveur à distance, tout en bloquant les accès non nécessaires. C'est une mesure de sécurité essentielle pour limiter les risques d'intrusion.

```
sudo ufw allow OpenSSH
sudo ufw enable
```

4. Supprimer les services inutiles

Sur un **VPS Debian 12** fraîchement installé, certains services peuvent être présents par défaut

Cette commande permet de **désinstaller les services** qu'on **n'utilise pas**, pour **libérer des ressources** et **réduire les risques de sécurité.**

```
sudo apt remove apache2 exim4 vsftpd -y
```

5. Installer les outils essentiels

Cette commande installe des **outils essentiels** pour gérer le serveur et faciliter le déploiement de l'application.

- **git** : pour cloner le projet depuis **GitHub**
- **curl** : pour tester les connexions **HTTP**
- **unzip** : pour extraire des archives si besoin
- **htop** : pour surveiller les ressources du serveur (CPU, RAM)

Ces outils sont légers, pratiques et indispensables pour travailler efficacement sur le VPS.

```
sudo apt install git curl unzip htop -y
```

6. Installation de Docker

Cette commande installe **Docker** et **Docker Compose**, ils sont installés dés la configuration initiale du VPS pour préparer l'environnement technique.

Cela permet de lancer les conteneurs, automatiser le déploiement et isoler les services de l'application qui se trouve dans son propre conteneur.

```
sudo apt install docker.io docker-compose -y

sudo usermod -aG docker stockmeatsolutions
```

On ajoute ensuite l'utilisateur **stockmeatsolutions** au groupe Docker pour qu'il puisse exécuter les commandes Docker **sans passer par sudo à chaque fois**, ce qui rend l'utilisation plus fluide.

Les vérifications et les réglages supplémentaires (tests des versions de Docker, activation du service) sont réalisés plus loin dans la section **SSH**.

7. Tester les accès et redémarrer le VPS

Cette commande permet de redémarrer complètement le VPS après avoir effectué toutes les configurations. Cela permet de valider que les modifications sont bien prises en compte (pare-feu, utilisateurs, services installés ou supprimés) et que le serveur redémarre sans erreur.

C'est une étape de **vérification** pour s'assurer que tout fonctionne correctement avant de passer à la suite.

```
sudo reboot
```

8. Installation de Java et Maven

Il faut penser aussi à installer **Java 17** et **Maven** afin compiler et exécuter une application **Spring Boot**

```
sudo apt install openjdk-17-jdk maven -y
```

Une fois le serveur bien préparé, sécurisé et équipé des outils nécessaires, il faut maintenant **mettre en place l'accès à distance avec SSH**. Cette étape permet de se connecter au serveur facilement et en toute sécurité, sans avoir à taper de mot de passe à chaque fois.

Grâce à cette configuration, on peut gérer le serveur à distance pour installer l'application, faire des mises à jour ou corriger des problèmes. Le système est maintenant propre, léger et prêt à accueillir **StockMeat Solutions.**

La configuration **SSH** vient compléter cette préparation en assurant un accès fiable et sécurisé au serveur.

8.2 Sécurisation de l'accès distant via SSH

SSH (Secure Shell) est **un protocole de communication sécurisé** qui permet de se connecter à un serveur à distance. Il chiffre les échanges entre le **PC** et le **VPS**, ce qui garantit la confidentialité des données transmises.

C'est l'outil standard pour diriger un **serveur Linux à distance**, tout en gardant un haut niveau de sécurité.

- **Étapes de configuration du SSH :**

1. **Générer une paire de clés SSH**

Cette commande crée une **clé RSA,** c'est une **paire de clé** utilisée pour **crypter** les données avec :

- Une **clé publique** pour **chiffrer les données.**
- Une **clé privée** pour les **déchiffrer.**

```
ssh-keygen -t rsa -b 4896 -f github-deploy-key -N ""
```

RSA est principalement utilisé afin de **sécuriser** les **connexions SSH**, de plus cette clé est de **4096 bits**, le choix de la **taille** de la clé **(en bits)** sert à déterminer le **niveau de sécurité**.

Plus la clé est **longue**, plus elle est **difficile à casser**, cela est indispensable pour rendre la clé plus **robuste** et c'est généralement suffisant.

2. Vérifier la création des fichiers

On liste les fichiers pour s'assurer que la **clé privée** (**github-deploy-key**) et la **clé publique** (**github-deploy-key.pub**) ont bien été générées.

```
ls -la
```

3. Copier la clé publique sur le serveur VPS

Cette commande envoie la **clé publique** sur le **VPS** pour autoriser l'accès **sans mot de passe**. Il faut entrer une dernière fois le mot de passe de l'utilisateur **stockmeatsolutions.**

```
ssh-copy-id -i github-deploy-key.pub stockmeatsolutions@180.149.196.2
```

L'adresse **IP** du **VPS (180.149.196.233)** est copiée directement depuis le **panel LWS**, car c'est **l'identifiant** du **VPS** qui permet de joindre le serveur à distance. Cela permet de cibler précisément le bon serveur.



4. Vérifier l'implémentation de Docker et finaliser la configuration

- Vérification de l'installation de Docker

Ces commandes permettent de vérifier que **Docker** et **Docker Compose** sont bien installés et reconnus par le système :

```
docker --version
docker compose --version
```

- Finalisation de la configuration Docker

Ces commandes permettent d'activer le service Docker, de s'assurer que l'utilisateur actuellement connecté via SSH dispose bien des droits nécessaires en l'ajoutant à Docker, et de vérifier que le moteur fonctionne correctement en listant les conteneurs actifs.

```
sudo usermod -aG docker $USER
sudo systemctl start docker
sudo systemctl enable docker
sudo docker ps
```

5. Créer le dossier de projet sur le serveur

On crée un dossier dédié à l'application et on donne les bons droits à l'utilisateur pour y accéder et y déployer les fichiers.

```
sudo mkdir /opt/stockmeatsolutions/Project-Apply
sudo chown -R $USER : $USER /opt/stockmeatsolutions/Project-Apply
```

6. Afficher la clé privée

Cette commande permet d'afficher le contenu de la clé privée, afin de la copier et la coller dans les variables d'environnements de **Github Actions.**

```
cat github-deploy-key
```

Une fois l'accès SSH sécurisé et le serveur prêt à accueillir l'application, il reste une dernière étape pour rendre **StockMeat Solutions** accessible publiquement : remplacer l'**adresse IP** par **un nom de domaine.**

Lors de l'achat du VPS sur la plateforme LWS, le nom de domaine était offert pendant un mois. Cette offre est suffisante pour présenter mon application avec une véritable adresse web, ce qui facilite l'accès et donne une image plus professionnelle au projet.

8.2 Configuration du nom de domaine pour accéder à l'application

- Étapes de configuration dans la zone DNS

Dans le panneau de gestion LWS, j'ai modifié les enregistrements DNS pour relier le nom de domaine à mon serveur VPS :

1. Enregistrement des adresses :

@	180.149.196.233	**Adresse principale du site**
www	180.149.196.233	**Version avec www**

Cela signifie que **stockmeatsolutions. site** et **www.stockmeatsolutions.site** pointent vers mon **VPS.**

2. Enregistrements des sous-domaines :

Pour que l'application puisse envoyer des mails via le service **Brevo** (la réinitialisation du mot de passe et l'envoi de facture), j'ai ajouté des redirections de sous-domaines vers les serveurs de Brevo. Ces enregistrements permettent à Brevo de **vérifier que le domaine m'appartient** et de **signer les mails** correctement, pour assurer une communication sécurisée et fiable entre le serveur VPS et Brevo.

brevo1._domainkey	b1.stockmeatsolutions-site.dkim.brevo.com
brevo2._domainkey	b2.stockmeatsolutions-site.dkim.brevo.com
smtp	mail.stockmeatsolutions.site.

Pour que Brevo puisse envoyer des mails en mon nom de manière sécurisée, j'ai ajouté d'autres paramètres dans la zone DNS. Ils permettent de **valider mon domaine**, de **signer les messages** pour prouver qu'ils viennent bien de moi, et de **limiter les risques de spam.**

@	brevo-code:46be5471b5b711ff5dacd64926744020
dkim._domainkey	v=DKIM1; k=rsa; p=MIGfMA0GCSqGSIb3DQEBAQUA...
_dmarc	v=DMARC1; p=none; rua=mailto:rua@dmarc.brev...

8.2.1 Sécurisation de l'accès avec SSL et proxy

Une fois le nom de domaine relié au serveur, il est important de sécuriser les échanges entre les **utilisateurs** et l'**application**. Cela passe par deux éléments : le **protocole HTTPS (via SSL)** et la gestion des **headers proxy**.

- **Le rôle du SSL (HTTPS)**

Le **SSL** (Secure Sockets Layer) permet de chiffrer les données échangées entre le navigateur et le serveur. Quand SSL est activé, l'application utilise le protocole **HTTPS** au lieu de HTTP, ce qui **protège les informations sensibles** (mots de passe, factures, etc.).

Dans mon fichier **application. properties**, j'ai préparé la configuration suivante :

```
# SSL
server.port=8080
server.ssl.enabled=false
server.ssl.key-store=/home/stockmeatsolutions/keystore.p12
server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD}
server.ssl.key-password=${SSL_KEYSTORE_PASSWORD}
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

Le **certificat SSL** est stocké dans un fichier sécurisé appelé **keystore. p12** et activable à tout moment. Les mots de passe sont injectés via des variables d'environnement pour éviter de les exposer dans le code. ***(Voir section 7.2 pour les détails sur la gestion des variables d'environnement.)***

- **Le rôle du proxy et des headers**

Quand l'application est hébergée derrière un proxy comme c'est le cas ici, ce proxy gère parfois le **HTTPS** à sa place. Pour que Spring Boot comprenne que les connexions sont sécurisées, il faut activer la gestion des **headers HTTP** transmis par le proxy en configurant également le fichier **application. properties** :

```
# Proxy / header
server.forward-headers-strategy=framework
```

Cette option permet à l'application de savoir si le visiteur utilise une **connexion sécurisée HTTPS**.

Ainsi, la **mise en ligne** de l'application est désormais opérationnelle : le serveur **VPS** est configuré, l'accès **SSH** est sécurisé, et le **nom de domaine** est relié avec prise en charge du **HTTPS** et des **proxys**. L'environnement est prêt pour accueillir les utilisateurs en toute sécurité.

Après avoir détaillé l'ensemble des fonctionnalités et de la logique métier côté serveur, nous pouvons désormais nous intéresser à la partie interface utilisateur, qui permet aux employés d'interagir avec l'application.

En somme après le **Backend** on passe au **Frontend.**

9 Interface Utilisateur

Dans cette section, nous allons voir ce qui a permis de concevoir les différentes pages web.

L'interface utilisateur repose sur les technologies listées dans la section "Technologies utilisées" du cahier des charges : **Thymeleaf**, **HTML**, **CSS** et **JavaScript**.

Thymeleaf est un moteur de Template intégré à **Spring Boot** qui permet d'**insérer dynamiquement des données** dans les **balises HTML,** il facilite la **liaison** entre le **backend (contrôleurs, services)** et le **frontend**.

Voici un exemple, pour illustrer le concept :

- **Exemple concret :** affichage du stock produit

Dans le contrôleur **ProductController**, on a une méthode qui se nomme **listProducts ()** qui récupère les produits en base de données, calcule le stock total et le nombre de scans fait par l'utilisateur, puis transmet ces données à la **vue** c'est-à-dire à la **page HTML** afin que celle-ci les **affiches** :

```
model.addAttribute("products", productService.getAllProducts());
model.addAttribute("stockTotal", productService.getTotalStock());
model.addAttribute("scanTotal", scans.size());
```

Contrôleur : ProductController



```
<tr th:each="product : ${products}">

  <td th:text="${product.name}"></td>
  <td th:text="${product.category}"></td>
  <td th:text="${product.weight} + 'g'"></td>
  <td th:text="${product.price} + '€'"></td>
  <td th:text="${product.quantity}"></td>

</tr>
```

Page Web : products.html

Pour chaque produit transmis par le contrôleur, affiche une ligne dans le tableau avec son nom, sa catégorie, son poids, son prix et sa quantité.

La balise **th : each** sert à **afficher automatiquement une série d'éléments** dans la page, comme la liste de produits. Elle évite d'écrire le même bloc **HTML** plusieurs fois : **une seule ligne suffit pour afficher tous les produits**, en répétant le modèle pour chacun.

Et la balise **th :text** : insère du texte dynamique dans une **balise HTML**, après la mise en place des subtilités de Thymeleaf pour le dynamisme des données dans le HTML.

Le reste de l'interface reste classique, avec du **CSS** pour le **style**, et du **JavaScript** pour les petites **interactions côté utilisateur**.

Le JavaScript a été utilisé à **trois reprises** dans le projet :

– pour la **détection du scanner**, ***(Voir section 6.4.3 Configuration du scanner)***

– pour **vérifier que les mots de passe correspondent** lors de la réinitialisation,

– et pour **réinitialiser automatiquement les champs** du formulaire de connexion à l'ouverture de la page.

L'ensemble des fonctionnalités côté backend ainsi que l'interface utilisateur sont désormais entièrement opérationnels. L'application est complète, à la fois robuste côté serveur et facile à utiliser côté client.

Nous pouvons maintenant nous pencher sur la phase de **tests**, qui permet de vérifier que tout fonctionne correctement, que les données sont bien protégées.

10. Tests

Les tests sont une étape essentielle dans le développement d'une application.
Dans le monde professionnel, ils permettent de **vérifier que chaque partie du code fonctionne correctement**, d'**éviter les bugs**, et de **gagner du temps** lors des mises à jour.

Une application bien testée est plus **fiable**, plus **sécurisée**, et plus **facile à maintenir**.

Dans ce projet, deux types de tests ont été réalisés : **les tests unitaires** et **les tests d'intégration**.

10.1 Outils utilisés

Pour réaliser les tests, plusieurs outils ont été utilisés :

- **JUNIT 5**

C'est l'outil de base pour écrire et exécuter les tests en Java, il permet de créer des méthodes de tests, de vérifier les résultats attendus mais aussi d'organiser les tests correctement.

Utilisé pour tous les tests, aussi bien unitaires qu'intégration.

- **Mockito**

C'est un outil qui permet de simuler le comportement d'un composant (comme un service ou un repository). On peut créer des faux objets pour tester une méthode sans dépendre des autres parties du code et de son fonctionnement.

Utilisé surtout dans les tests unitaires.

- **MockMvc**

C'est un outil fourni par Spring pour tester les contrôleurs web sans lancer le serveur. Il permet de simuler des requêtes HTTP (GET, POST…) et de vérifier les réponses.

Utilisé dans les tests d'intégration.

10.1 Tests unitaires

Les tests unitaires servent à **vérifier le bon fonctionnement d'une méthode ou d'un composant précis**, de manière isolée.
Par exemple, on teste une méthode de calcul, une fonction de validation, ou un service

Dans ce projet, les tests unitaires ont été réalisés sur :

- les **services** (ex : ProductService, UserService, InvoiceService),
- les **mappers** (ex : ScanMapper, UserMapper),
- les **repositories** (ex : UserRepositoryTest).

Ces tests permettent de s'assurer que chaque bloc de logique fonctionne comme prévu, **même si le reste de l'application n'est pas lancé**.

Cela veut dire qu'on peut tester une méthode ou un service **tout seul**, sans avoir besoin de démarrer le serveur, la base de données, ou l'interface utilisateur.

10.2 Tests d'intégration

Les tests d'intégration permettent de **vérifier que plusieurs parties du code fonctionnent bien ensemble**.
On teste ici les **contrôleurs**, les **routes HTTP**, les **interactions avec la base de données**, et les **règles de sécurité**.

Pour cela, une **base de données H2** en mémoire a été utilisée car elle est rapide, légère et ne garde pas les données après le test. Cela permet de tester le fonctionnement du code sans toucher à la vraie base, et de repartir à zéro à chaque test, ce qui évite les erreurs liées aux anciennes données.

Par conséquent, on est obligé de créer un fichier de configuration spécial

(application-test. properties) pour l'exécution des tests d'intégration

Application-test. properties

```
# Datasource
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
```

Les tests d'intégration ont été réalisés sur :

- les **contrôleurs** (ex : ProductControllerTest, ScanControllerTest, LoginControllerTest),
- les **services liés à la sécurité** (ex : CustomUserDetailsServiceTest),
- les **repositories** (ex : UserRepositoryIT).

Ces tests permettent de **valider les scénarios complets**, comme la connexion, l'ajout de produit, ou le paiement. L'ensemble des tests unitaires et d'intégration est disponible dans le projet. Des extraits représentatifs sont présentés en annexe. ***(Annexe 25)***

10.3 Rapports de couverture

Pour mesurer la qualité des tests, deux outils ont été utilisés :

- **JaCoCo** : il calcule le **taux de couverture du code**, c'est-à-dire combien de lignes et de branches ont été testées.
- **Surefire** : il affiche un **résumé clair des résultats des tests** (nombre de tests, erreurs, succès, durée…).

Ces rapports montrent que les tests sont bien en place, avec un **taux de réussite de 100%** et une **couverture globale supérieur à 80%**, ce qui est généralement considérer comme un bon niveau de couverture en milieu professionnel.

Des extraits représentatifs des rapports JaCoCo et Surefire sont présentés en annexe, afin d'illustrer la couverture de code et les résultats des tests. ***(Annexe 26)***

Grâce aux différents tests, le bon fonctionnement de l'application à été vérifier avec chaque partie du code et l'interaction avec l'ensemble du code et la base de données.

Il ne reste plus qu'une dernière étape pour finaliser le projet : la **documentation**, indispensable pour guider les utilisateurs mais aussi les développeurs. Il ne reste plus qu'a tester

11. Documentation

La documentation est l'ultime étape du projet, mais elle est toute aussi importante.

Elle permet d'autres personnes (développeurs, utilisateurs) de comprendre comment fonctionner l'application avec ses différentes classes et les méthodes qu'elles contiennent.

11.1 JavaDoc

Pour documenter le code Java, j'ai utilisé **JavaDoc**.

C'est à outil intégrer au langage Java qui permet de générer **automatiquement des pages web à partir des commentaires spéciaux écrits dans le code**.

Ces pages expliquent :

- à quoi sert chaque classe ou méthode,
- quels paramètres sont utilisé,
- et ce que la méthode retourne.

Cela rend le code **plus clair** et **plus facile à lire**, surtout pour les développeurs qui découvrent le projet.

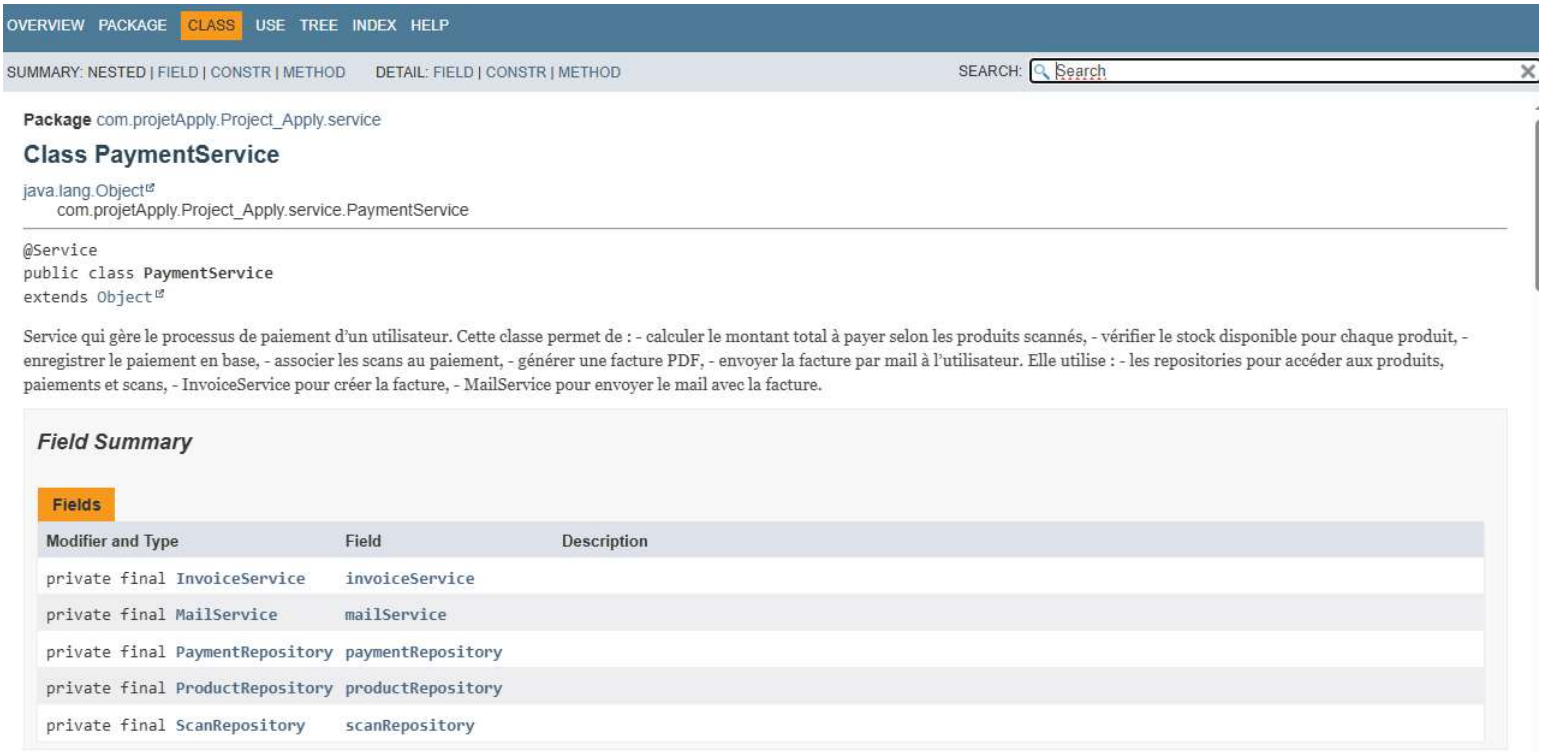
Comme par exemple cette classe de la couche service **PaymentService** :

voici les commentaires

```java
/**
 * Service qui gère le processus de paiement d'un utilisateur.
 *
 * Cette classe permet de :
 * - calculer le montant total à payer selon les produits scannés,
 * - vérifier le stock disponible pour chaque produit,
 * - enregistrer le paiement en base,
 * - associer les scans au paiement,
 * - générer une facture PDF,
 * - envoyer la facture par mail à l'utilisateur.
 *
 * Elle utilise :
 * - les repositories pour accéder aux produits, paiements et scans,
 * - InvoiceService pour créer la facture,
 * - MailService pour envoyer le mail avec la facture.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class PaymentService {
```

Ce qui me génère cette page web :

on peut voir les commentaires qui apparaissent sur la page et les différentes méthodes de la classe



OVERVIEW PACKAGE CLASS USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH: Search

Package com.projetApply.Project_Apply.service

Class PaymentService

java.lang.Object
com.projetApply.Project_Apply.service.PaymentService

```
@Service
public class PaymentService
extends Object
```

Service qui gère le processus de paiement d'un utilisateur. Cette classe permet de : - calculer le montant total à payer selon les produits scannés, - vérifier le stock disponible pour chaque produit, - enregistrer le paiement en base, - associer les scans au paiement, - générer une facture PDF, - envoyer la facture par mail à l'utilisateur. Elle utilise : - les repositories pour accéder aux produits, paiements et scans, - InvoiceService pour créer la facture, - MailService pour envoyer le mail avec la facture.

Field Summary

Fields

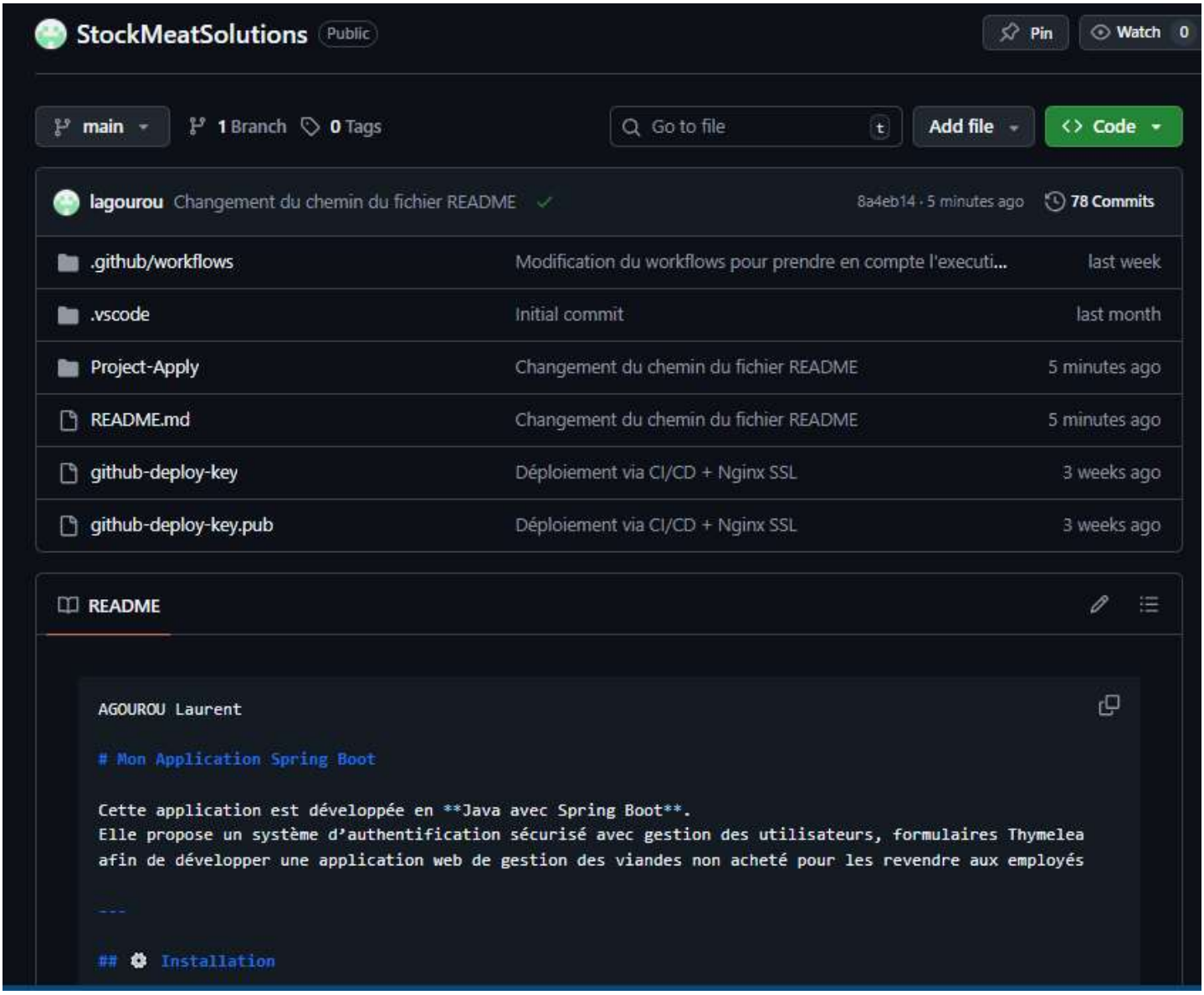
Modifier and Type	Field	Description
private final InvoiceService	invoiceService	
private final MailService	mailService	
private final PaymentRepository	paymentRepository	
private final ProductRepository	productRepository	
private final ScanRepository	scanRepository	

11.2 Github et README

J'ai aussi utilisé **GitHub** pour héberger le projet, et j'ai rédigé un fichier **README** pour expliquer :

- quelle est la nature de l'application à quoi elle sert et à qui
- comment **installer et lancer l'application**,
- quels **outils** sont nécessaires (Java, Maven, Spring Boot…),
- quelle est **l'architecture** du projet (backend, frontend, base de données…),
- quelles sont les fonctionnalités du projet,
- et comment faire les **tests** ou accéder à la **documentation technique**.

Ce fichier est très utile pour toute personne qui veut **utiliser** ou **contribuer** au projet. Voici ma page **GitHub** avec le **README** intégré.



12 Conclusion

12.1 Rappel de l'objectif du projet

Ce projet avait pour objectif de concevoir une application web sécurisée pour la gestion des stocks de viande, adaptée aux besoins d'une entreprise de transformation. L'idée était de proposer une solution simple, efficace et adaptée aux besoins réels du terrain.

12.2. Synthèse des fonctionnalités réalisées

L'application permet :

- de s'authentifier de manière sécurisée,
- de réinitialiser son mot de passe via un lien envoyé par mail
- de consulter et gérer les produits en stock via une interface claire,
- de suivre les produits scannés,
- et d'effectuer des paiements internes permettant à un utilisateur de recevoir sa facture PDF par mail

Elle est déployée dans un environnement Docker, automatisée avec GitHub Actions, et configurée pour fonctionner sur un VPS avec nom de domaine.

12.3 Méthodologie et bonnes pratiques

Le projet a été développé avec **Spring Boot**, en respectant une architecture claire basée sur le modèle **MVC (Modèle-Vue-Contrôleur)**. Cette structure permet de séparer la logique métier, la gestion des données et l'affichage, ce qui facilite la maintenance et l'évolution du code.

La couche frontend a été réalisée avec **Thymeleaf**, un moteur de Template intégré à Spring, qui permet de générer dynamiquement des pages HTML tout en gardant une syntaxe simple et intuitive.

Les environnements dev, test et vps ont été configurés avec des profils adaptés, permettant de tester et déployer l'application dans des contextes différents. L'accent a été mis sur la lisibilité du code, et la facilité de maintenance, avec des tests unitaires et d'intégration, ainsi qu'une documentation synthétique et accessible.

12.4 Ce que j'ai appris

Ce projet m'a permis de :

- Configurer un vrai scanner de code-barres pour une utilisation concrète.
- Définir les besoins des utilisateurs en fonction de mon expérience terrain dans le secteur de la viande.
- Structurer le code et la documentation pour qu'ils soient clairs, simples et faciles à assimiler.
- Sécuriser l'application, tester les fonctionnalités, et préparer un déploiement professionnel.
- Comment configurer l'envoi automatique de mail

12.5 Ouverture et perspectives

Ce projet peut évoluer vers une version commerciale, dédiée à une entreprise spécialisée dans la découpe de viande. Il pourrait intégrer un vrai système de paiement en ligne (Visa, PayPal, etc.), et un tableau de bord analytique.

Ce tableau de bord serait une page qui affiche des chiffres et des graphiques pour suivre l'état du stock, les paiements et l'activité des utilisateurs. Il aiderait l'entreprise à mieux gérer son travail au quotidien et à réduire le gaspillage, en repérant rapidement les produits en surplus ou en rupture.

Annexes :

Annexe 1 : Kanban..... - 63 -
Annexe 2 : Insert des données..... - 65 -
Annexe 3 : Application.properties..... - 66 -
Annexe 4 : Entités..... - 68 -
Annexe 5 : Authentification avec Spring Security..... - 71 -
Annexe 6 : Réinitialisation du mot de passe par email..... - 74 -
Annexe 7 : Code source du service d'envoi de mail..... - 75 -
Annexe 8 : Classes DTO utilisées pour le transfert de données..... - 76 -
Annexe 9 : Classes de gestion des utilisateurs (services et contrôleurs)..... - 78 -
Annexe 10 : Pages de connexion, d'inscription et profil..... - 82 -
Annexe 11 : Classes de gestion des *produits* (services et contrôleurs)..... - 84 -
Annexe 12 : Interface de gestion des produits..... - 87 -
Annexe 13 : Gestion du scan et de la traçabilité (service et contrôleur)..... - 88 -
Annexe 14 : Page de la gestion des Scans..... - 89 -
Annexe 15 : Exemple de Code-barres..... - 90 -
Annexe 16 : Branchement du Scanner..... - 91 -
Annexe 17 : Script AutoHotKey pour l'automatisation du scanner (scanner.ahk) - 92 -
Annexe 18 : Notice d'utilisation du Scanner..... - 93 -
Annexe 19 : Interface de la gestion de Paiement..... - 95 -
Annexe 20 : Exemple de Facture générée (PDF)..... - 96 -
Annexe 21 : Mail reçue avec Facture en pièce jointe..... - 97 -
Annexe 22 : Fichier .env..... - 98 -
Annexe 23 : Fichier docker-compose.yml..... - 99 -
Annexe 24 : Commande Docker..... - 100 -
Annexe 25 : Exemples de tests unitaires et d'intégration (ProductServiceTest & ProductRepositoryIT)..... - 101 -
Annexe 26 : Rapports JaCoCo et Surefire..... - 103 -

Annexe 1 : Kanban

Kanban StockMeat Solutions

Tâches	Status	Rôle	Description	Critère d'acceptation	Priorité
Inscription utilisateur	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir m'inscrire avec mon email, un nom d'utilisateur et un mot de passe afin d'accéder à l'application.	Compte créé avec succès	Très Haute
Connexion/ Authentification	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir me connecter avec mon email et mot de passe pour accéder à mon compte.	Session valide, pages protégées accessibles après connexion	Très Haute
Réinitialisation mot de passe	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite recevoir un email avec un lien pour réinitialiser mon mot de passe en cas d'oubli.	Email envoyé via Brevo, token avec durée d'expiration	Haute
CRUD produit	Terminé	Développeur	En tant que développeur, je souhaite pouvoir ajouter, modifier, supprimer et lister les produits dans le système afin de permettre une gestion complète du stock	CRUD fonctionne et les validations des données sont implémentées	Haute
Gestion de stock	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite voir le statut du stock de chaque produit et le stock total	Changement du statut du stock dès qu'un produit est en rupture de stock ou un stock faible, affichage du stock total	Très Haute
Scan produit	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite scanner un code-barres via un scanner sans fil ou taper directement le code pour identifier rapidement un produit.	Code-barres EAN13 scanné via scanner enregistré dans la BDD, champ input reçoit le code	Très Haute

Enregistrement paiement	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir enregistrer un paiement lié à un ou plusieurs produits scannés, afin de finaliser la transaction et vider le panier.	Paiement déclenché uniquement si les produits sont scannés et ajouter au panier. Montant total correctement calculé. Panier vidé après validation du paiement	Très Haute
Génération facture PDF	Terminé	Utilisateur	En tant qu'utilisateur je souhaite recevoir par mail un PDF de facture	PDF valide généré avec les produits scannés, la quantité , le prix et le total, email envoyé via Brevo	Haute
Conteneurisation de l'application	Terminé	Développeur	En tant que développeur, je souhaite mettre en place une conteneurisation de l'application afin de faciliter le déploiement, le partage et le bon fonctionnement sur différents environnements	Un Dockerfile permet de créer une image fonctionnelleL'application démarre correctement dans un conteneurLe port 8080 est bien exposéLe conteneur fonctionne sur plusieurs machines sans modification	Haute
Déploiement sur VPS avec nom de domaine et accès SSH	Terminé	Développeur	En tant que développeur, je souhaite déployer l'application sur un serveur VPS avec un nom de domaine configuré et un accès SSH sécurisé, afin de rendre le service accessible en ligne et administrable à distance.	Le VPS est opérationnel et accessible via SSHLes clés SSH sont configurées pour un accès sécuriséLe nom de domaine pointe vers l'IP du serveurL'application est déployée et accessible via le domaineLes ports nécessaires sont ouverts et protégés	Haute
Configuration de la base de données	Terminé	Développeur	En tant que développeur, je souhaite configurer la base de données MySQL afin de stocker et interroger les données produits, scans, utilisateurs et paiements de manière fiable.	La BDD est créée et accessibleLes entités JPA sont correctement mappées Les relations (OneToMany, ManyToOne...) sont fonctionnellesLes données sont persistées et récupérées sans erreur Le schéma MPD est documenté	Très Haute
Mise en place du CI/CD avec GitHub Actions	Terminé	Développeur	En tant que développeur, je souhaite automatiser le build, les tests et le déploiement de l'application via GitHub Actions afin de sécuriser et faciliter la mise en ligne rapide des nouvelles modifications apportées aux codes	Un workflow `.github/workflows/ci-cd.yml` est en place Le build Maven est automatisé Les tests unitaires sont lancés à chaque push et l'application est déployé automatiquement dans des conteneurs Docker hébergés sur le VPS Les erreurs sont visibles dans GitHub	Haute
Implémentation des tests unitaires et d'intégration	Terminé	Développeur	En tant que développeur, je souhaite écrire des tests unitaires et d'intégration pour valider le bon fonctionnement de l'application.	Les tests unitaires couvrent les services métier (stock, paiement, facture) et autres service pour la couverture de test Les tests d'intégration vérifient les endpoints REST La couverture de code est mesurée avec Jacoco, et les rapports de tests sont générés via le plugin Maven Surefire Les tests sont lancés automatiquement via CI	Haute
Documentation du projet	Terminé	Développeur	En tant que développeur, je souhaite rédiger un README clair sur GitHub et générer la Javadoc du projet afin de faciliter la compréhension du code et son utilisation par d'autres développeurs	Un fichier `README.md` est présent à la racine du projetLe README contient : présentation, installation, usage, endpoints, déploiement La Javadoc est générée pour les classes et services principaux Les deux sont accessibles et à jour sur GitHub	Haute
Interface utilisateur	Terminé	Utilisateur	En tant qu'utilisateur, je souhaite pouvoir interagir avec l'application en utilisant une interface web qui est possède une certaine esthétique, qui est responsive et intuitive	Un site web avec plusieurs pages pour voir la liste des produits sous formes de tableau avec le statut du stock , des formulaires pour se connecter, pour s'inscrire pour pouvoir changer les informations personnels de l'utilisateur, une page pour le scan avec un champ de saisie pour entrer le code de barre d'un produit	Très Haute

Annexe 2 : Insert des données

```
mysql> SELECT * FROM user;
ERROR 1146 (42S02): Table 'project_apply.user' doesn't exist
mysql> SELECT * FROM users;
+----+--------------------+-----------+--------------------------------------------------------------+--------------------------------------+---------------------+
| id | email              | username  | password                                                     | reset_token                          | token_expiration    |
+----+--------------------+-----------+--------------------------------------------------------------+--------------------------------------+---------------------+
|  4 | agourou84@gmail.com | agourou84 | $2a$10$MmWc/UIutTAxl4MGAlWI0.crqc.eq8B3QQi2BsLkTgQSxwBTPB5YG | 23ae40c8-19b2-4eef-bd4a-87b6d0708c6e | 2025-08-26 08:00:06 |
+----+--------------------+-----------+--------------------------------------------------------------+--------------------------------------+---------------------+
```

Insertion des données de la table Users

```
mysql> SELECT * FROM product;
+----+-------------------+---------------+----------+--------+-------+------------------+----------------+
| id | name              | barcode       | quantity | weight | price | status           | category       |
+----+-------------------+---------------+----------+--------+-------+------------------+----------------+
|  1 | Entrecôte de b?uf | 1234567890128 |       15 |    300 |  2.00 | Stock moyen      | Viande rouge   |
|  2 | Blanc de poulet   | 9876543210982 |       40 |    200 |  1.50 | Stock disponible | Volaille       |
|  3 | Côte de porc      | 4567890123456 |        5 |    350 |  2.50 | Stock faible     | Viande blanche |
|  4 | Dinde             | 7958920147202 |        0 |    100 |  1.00 | Rupture de stock | Volaille       |
+----+-------------------+---------------+----------+--------+-------+------------------+----------------+
4 rows in set (0.00 sec)
```

Insertion des données de la table Product

Annexe 3 : Application.properties

```
spring.application.name=Project-Apply

# Logging
logging.level.org.springframework=DEBUG
logging.level.org.springframework.web=DEBUG
logging.level.org.springframework.security=DEBUG
logging.level.org.springframework.security.web=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type=TRACE
logging.level.org.springframework.mail=DEBUG
logging.level.org.springframework.boot.autoconfigure.mail.MailSenderAutoConfiguration=DEBUG
logging.level.com.projetApply.Project_Apply=INFO

# Jpa
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.jdbc.batch_size=20
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
```

```
# Thymeleaf
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.check-template-location=true
spring.thymeleaf.cache=false
spring.thymeleaf.enabled=true

# Mail
spring.mail.host=smtp-relay.brevo.com
spring.mail.port=587
spring.mail.username=9583ad001@smtp-brevo.com
spring.mail.password=${SPRING_MAIL_PASSWORD}
spring.mail.protocol=smtp
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.debug=true
```

```
# SSL
server.port=8080
server.ssl.enabled=false
server.ssl.key-store=/home/stockmeatsolutions/keystore.p12
server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD}
server.ssl.key-password=${SSL_KEYSTORE_PASSWORD}
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat

# Proxy / header
server.forward-headers-strategy=framework

# Profil
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}
```

Annexe 4 : *Entités*

```java
@Entity
@Table(name = "Payment")
@Data
public class Payment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    private PaymentType type;

    @Column(name = "payment_date")
    private LocalDateTime paymentDate;

    @ManyToOne
    @JoinColumn(name = "employee_id")
    private User employee;

}
```

Entité de Paiement

```java
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "barcode")
    private String barcode;

    @Column(name = "quantity")
    private int quantity;

    @Column(name = "weight")
    private int weight;

    @Column(name = "price", precision = 10, scale = 2)
    private BigDecimal price;

    private String status;

    private String category;

    @OneToMany(mappedBy = "product")
    private List<Scan> scans;
```

Entité de Produit

```java
public class Scan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(name = "date_scan", nullable = false)
    private Timestamp dateScan;

    @ManyToOne
    @JoinColumn(name = "payment_id")
    private Payment payment;

}
```

Entité de Scan

Annexe 5 : Authentification avec Spring Security

```
public class SecurityConfig {

	private final CustomUserDetailsService customUserDetailsService;

	@Bean
	public PasswordEncoder passwordEncoder() {
		log.info("Initialisation du mot de passe encoder");
		return new BCryptPasswordEncoder();
	}

	@Bean
	public AuthenticationManager authenticationManager(HttpSecurity http, PasswordEncoder passwordEncoder)
			throws Exception {
		log.info("Configuration de l'Authentication Manager");
		AuthenticationManagerBuilder authenticationManagerBuilder = http
				.getSharedObject(AuthenticationManagerBuilder.class);
		authenticationManagerBuilder.userDetailsService(customUserDetailsService)
				.passwordEncoder(passwordEncoder);

		return authenticationManagerBuilder.build();
	}

	@Bean
	public WebSecurityCustomizer webSecurityCustomizer() {
		return (web) -> web.ignoring().requestMatchers("/api/status");
	}

	@Bean
	public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

		log.info("Définition des règles de sécurité");

		return http
				.authorizeHttpRequests(auth -> auth
						// Pages publiques
						.requestMatchers("/", "/login", "/register", "/forget-password",
								"/reset-password")
						.permitAll()
						.requestMatchers("/css/**", "/js/**", "/images/**").permitAll()
						.requestMatchers("/api/status").permitAll()
						// Pages nécessitant une authentification
						.requestMatchers(
								"/profil", "/modify-profil",
								"/form", "/add", "/remove", "/{id}",
								"/user/**", "/products/**", "/save",
								"/send-invoice", "/scans/**", "/payment/**")
						.authenticated()

						// Tout le reste est protégé
						.anyRequest().authenticated())
				.formLogin(form -> form
						.loginPage("/login")
						.usernameParameter("email")
						.passwordParameter("password")
						.defaultSuccessUrl("/profil", true) // redirection après login
						.permitAll())
				.logout(logout -> logout
						.logoutUrl("/logout")
						.logoutSuccessUrl("/login?logout")
						.permitAll())
				.csrf(csrf -> csrf.disable())
				.build();
```

```java
public class UserDetailsImplements implements UserDetails {

    private int id;
    private String username;
    private String email;
    private String password;

    public UserDetailsImplements(User user) {

        log.info("Création de l'objet UserDetails pour utilisateur ID: {}", user.getId());
        this.id = user.getId();
        this.username = user.getUsername();
        this.email = user.getEmail();
        this.password = user.getPassword();

    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return username;
    }

    public String getEmail() {
        return email;
    }

    public int getId() {
        return id;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        log.info("Aucune autorisation définie pour utilisateur ID: {}", id);
        return List.of();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
```

```java
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        log.info("Recherche utilisateur avec email: {}", email);

        User user = userRepository.findByEmail(email)
                .orElseThrow(
                        () -> new UsernameNotFoundException("L'utilisateur avec l'email " + email + " n'existe pas"));
        log.info("Utilisateur trouvé: {}", email);
        return new UserDetailsImplements(user);
    }

}
```

Annexe 6 : Réinitialisation du mot de passe par



Annexe 7 : Code source du service d'envoi de mail

```java
public class MailServiceImplements implements MailService {

    private final JavaMailSender mailSender;
    @Override
    public void sendMail(String to, String from, String subject, String body) {
        log.info("Préparation du mail : to={}, from={}, subject={}", to, from, subject);
        log.info("Mot de passe SMTP injecté : {}", System.getenv("SPRING_MAIL_PASSWORD"));

        MimeMessage message = mailSender.createMimeMessage();

        try {
            MimeMessageHelper helper = new MimeMessageHelper(message, true, "UTF-8");
            helper.setTo(to);
            helper.setFrom(from);
            helper.setSubject(subject);
            helper.setText(body, true);
            log.info("Message MIME configuré avec succès");

            log.info("Tentative d'envoi du mail via JavaMailSender...");
            mailSender.send(message);
            log.info("Mail envoyé avec succès à {}", to);

        } catch (MessagingException | MailException e) {
            log.error("Échec de l'envoi du mail à {}", to, e);

            throw new MailSendingException("Impossible d'envoyer le mail", e);
        }
    }

      @Override
    public void sendMailWithAttachment(String to, String from, String subject, String body, byte[] attachment,
            String filename) {
        MimeMessage message = mailSender.createMimeMessage();

        try {
            MimeMessageHelper helper = new MimeMessageHelper(message, true);
            helper.setTo(to);
            helper.setFrom(from);
            helper.setSubject(subject);
            helper.setText(body);

            ByteArrayResource resource = new ByteArrayResource(attachment);
            helper.addAttachment(filename, resource);

            mailSender.send(message);

        } catch (MessagingException e) {
            throw new RuntimeException("Erreur lors de l'envoi du mail avec pièce jointe", e);
        }
    }

}
```

```java
public interface MailService {
        void sendMail(String to, String from, String subject, String body);

        void sendMailWithAttachment(String to, String from, String subject, String body,
byte[] attachment,
            String filename);

}
```

Annexe 8 : Classes DTO utilisées pour le transfert

```java
public class UserDTO {

    private int id;

    @NotBlank(message = "Le nom d'utilisateur est requis")
    @Size(min = 8, max = 20, message = "Le nom d'utilisateur doit contenir entre 8 et 20 caractères")
    private String username;

    @NotBlank(message = "L'email est requis")
    @Email(message = "L'email doit être valide")
    private String email;

    @NotBlank(message = "Le mot de passe est requis")
    @Size(min = 6, max = 32, message = "Le mot de passe doit contenir entre 6 et 32 caractères")
    private String password;
}
```

```java
public class ProductDTO {

    private int id;

    private String name;

    private String barcode;

    private int quantity;

    private int weight;

    @DecimalMin(value = "0.1")
    @Digits(integer = 10, fraction = 2)
    @NumberFormat(style = NumberFormat.Style.NUMBER)
    private BigDecimal price;

    private String status;

    private String category;
}
```

```java
public class ScanDTO {

    private int id;

    private int userId;

    private String userName;

    private String codeBarre;

    private Timestamp dateScan;

    private ProductDTO product;

}
```

```java
public class PaymentDTO {

    private int id;

    @NotNull
    @DecimalMin("0.01")
    private BigDecimal amount;

    @NotNull
    private PaymentType type;

    private LocalDateTime paymentDate;

    @NotNull
    private String status;

    @Min(1)
    private int employeeId;

}
```

Annexe 9 : Classes de gestion des utilisateurs (services et

```java
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final UserMapper userMapper;

      public User getUserById(int userId) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new EntityNotFoundException("Utilisateur introuvable"));

        return user;
    }

      public User getUserByEmail(String email) {
        return userRepository.findByEmail(email)
                .orElseThrow(
                        () -> new UserNotFoundException("Utilisateur avec cet email " + email + "introuvable"));
    }

      public void saveUser(User user) {
        userRepository.save(user);
    }

      public void saveNewUser(UserDTO userDTO) {
        User user = userMapper.toEntity(userDTO);
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        userRepository.save(user);
    }

    public void deleteUser(int id) {
        User user = userRepository.findById(id)
                .orElseThrow(() -> new EntityNotFoundException("Utilisateur introuvable"));
        userRepository.delete(user);
    }

    public void updateUser(UserDTO updateUser, int id) {
        User user = getUserById(id);
        user = mergeUpdateUser(user, updateUser);
        saveUser(user);

        UserDetailsImplements updateDetails = new UserDetailsImplements(user);
        UsernamePasswordAuthenticationToken newAuth = new UsernamePasswordAuthenticationToken(
                updateDetails, updateDetails.getPassword(), updateDetails.getAuthorities());

        SecurityContextHolder.getContext().setAuthentication(newAuth);

    }

      private User mergeUpdateUser(User existingUser, UserDTO updateUserDTO) {

        Optional.ofNullable(updateUserDTO.getUsername())
                .filter(username -> isValidUpdateUser(username, existingUser.getUsername()))
                .ifPresent(existingUser::setUsername);

        Optional.ofNullable(updateUserDTO.getEmail())
                .filter(email -> isValidUpdateUser(email, existingUser.getEmail()))
                .ifPresent(existingUser::setEmail);

        Optional.ofNullable(updateUserDTO.getPassword())
                .filter(password -> !password.isBlank())
                .map(passwordEncoder::encode)
                .ifPresent(existingUser::setPassword);

        return existingUser;
    }

    private boolean isValidUpdateUser(String newValue, String existingValue) {
        return newValue != null && !newValue.isBlank() && !newValue.equals(existingValue);
    }
```

```java
public class LoginController {

    @GetMapping("/login")
    public String login() {
        log.info("Accès à la page de connexion");
        return "login";
    }

    @GetMapping("/")
    public String redirectToLogin() {
        return "redirect:/login";
    }
```

```java
public class RegisterController {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @GetMapping("/register")
    public String register(Model model) {

        log.info("Accès à la page d'inscription");
        model.addAttribute("user", new UserDTO());
        return "register";
    }

    @PostMapping("/register")
    public String registerUser(@Valid @ModelAttribute("user") UserDTO userDto,
            BindingResult result, Model model) {

        if (result.hasErrors()) {
            log.warn("Erreur dans le formulaire d'inscription");
            model.addAttribute("user", userDto);
            return "register";
        }

        if (userRepository.findByEmail(userDto.getEmail()).isPresent()) {

            log.warn("Email déjà utilisé : {}", userDto.getEmail());
            model.addAttribute("error", "Email déjà utilisé");
            return "register";
        }
        User user = new User();
        user.setUsername(userDto.getUsername());
        user.setEmail(userDto.getEmail());
        user.setPassword(passwordEncoder.encode(userDto.getPassword()));
        userRepository.save(user);

        log.info("Nouvel utilisateur inscrit : {}", userDto.getEmail());
        model.addAttribute("success", "Inscription réussie");
        return "redirect:/login";
    }
```

```java
public class ProfilController {

    private final UserService userService;
    private final UserMapper userMapper;

    @GetMapping("/profil")
    public String profil(@AuthenticationPrincipal UserDetailsImplements userDetails, Model model) throws Exception

        if (userDetails != null) {
            UserDTO userForm = userMapper.toDTO(userService.getUserById(userDetails.getId()));
            model.addAttribute("user", userForm);
        } else {
            log.warn("Utilisateur non authentifié, redirection vers la page de connexion.");
            return "redirect:/login";
        }
        return "profil";
    }

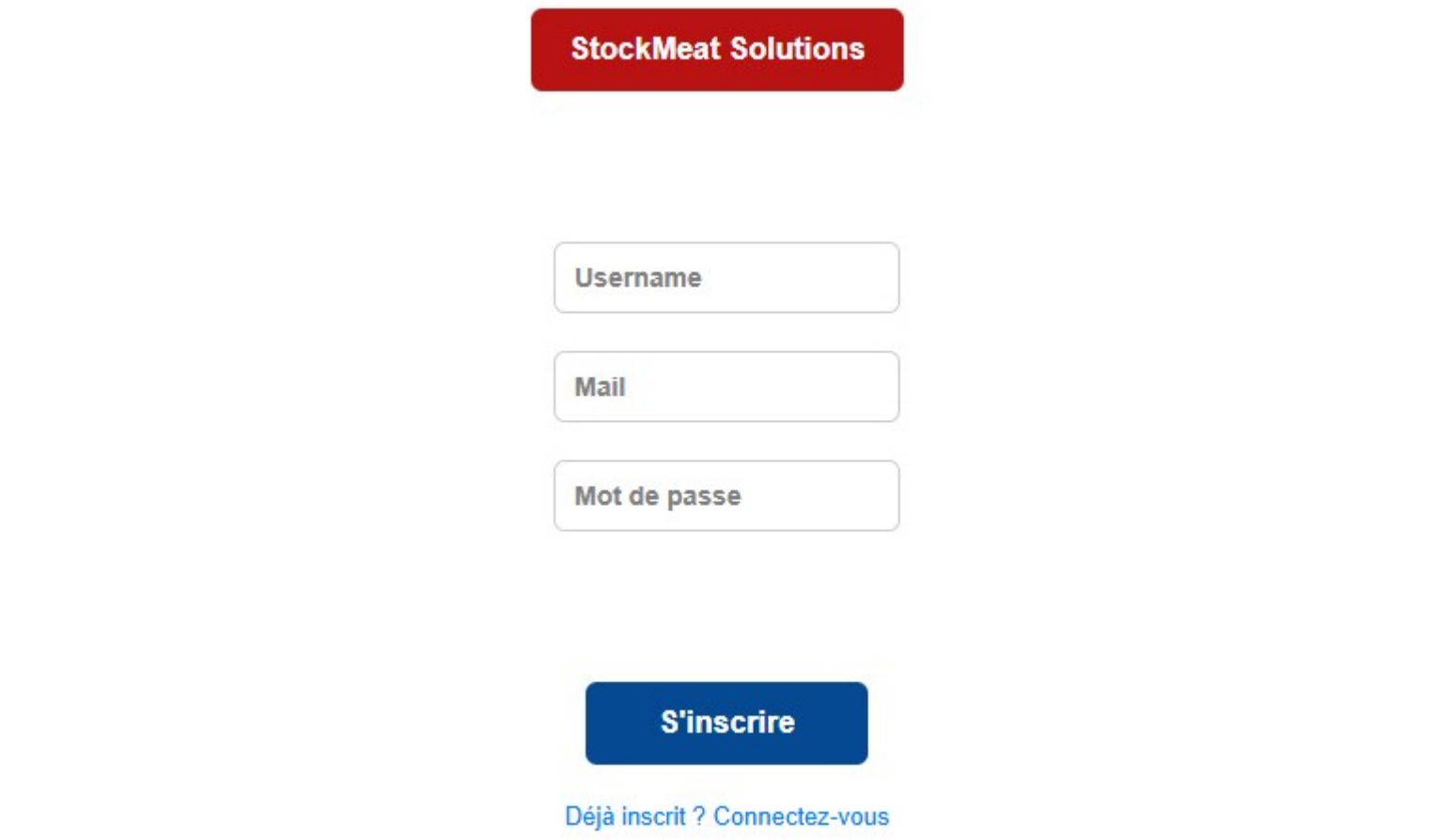
    @PostMapping("/modify-profil")
    public String modifyUser(@Valid @ModelAttribute("user") UserDTO updatedUser, BindingResult bindingResult,
            @AuthenticationPrincipal UserDetailsImplements userDetails, RedirectAttributes redirectAttributes)
            throws Exception {

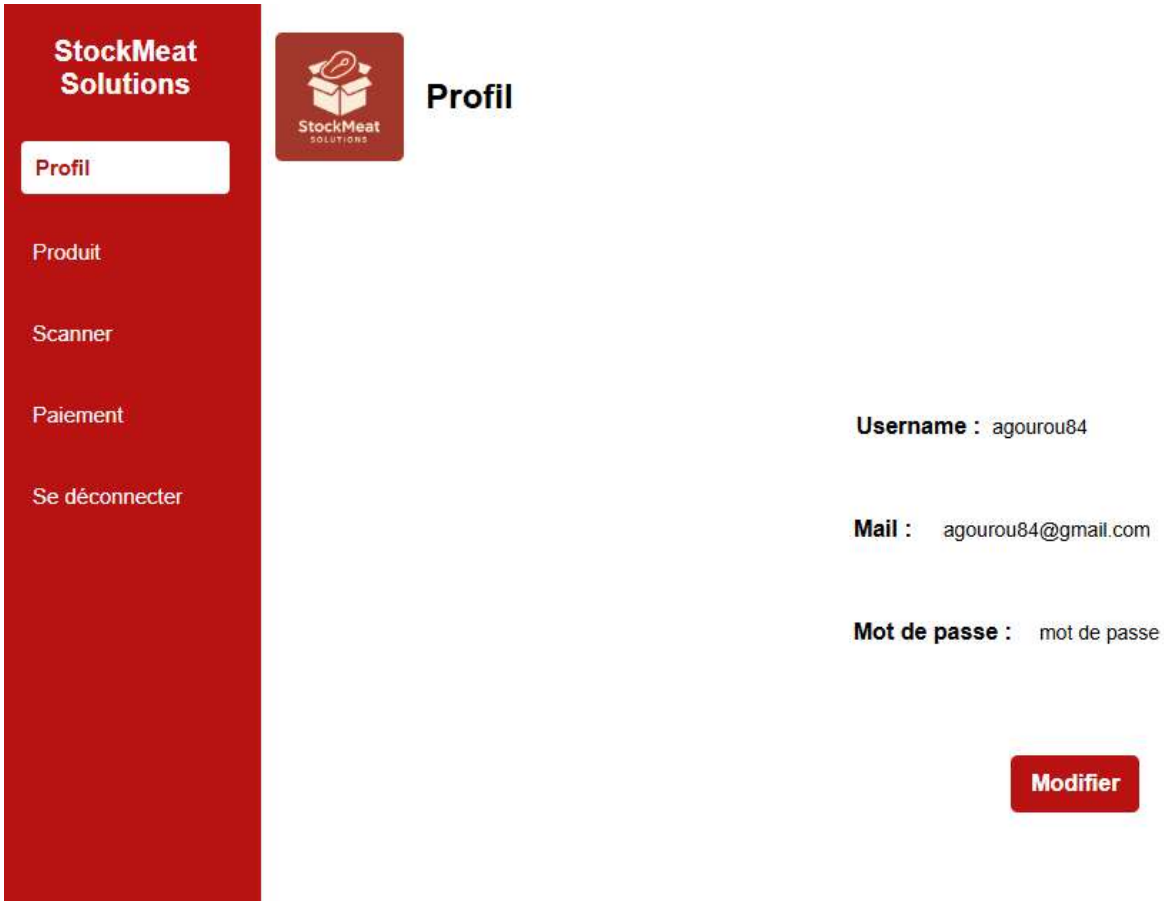
        if (bindingResult.hasErrors()) {
            log.warn("Erreur dans la modification du profil: {}", bindingResult.getAllErrors());
            return "profil";
        }
        log.info("Modification du profil de l'utilisateur ID: {}", userDetails.getId());
        userService.updateUser(updatedUser, userDetails.getId());
        redirectAttributes.addFlashAttribute("success", "✔ Profil bien modifié");

        return "redirect:/profil";
    }
```

Annexe 10 : Pages de connexion, d'inscription et profil






StockMeat Solutions
Profil
Produit
Scanner
Paiement
Se déconnecter
StockMeat
SOLUTIONS
Profil
Username : agourou84
Mail : agourou84@gmail.com
Mot de passe : mot de passe
Modifier

Annexe 11 : Classes de gestion des *produits* (services et

```java
public class ProductService {

    private final ProductRepository productRepository;

    private final ProductMapper productMapper;

   public ProductDTO addProduct(String barcode, String name, int quantite, int weight, BigDecimal
price) {
        Product product = productRepository.findByBarcode(barcode).orElse(null);

        if (product == null) {
            product = new Product();
            product.setBarcode(barcode);
            product.setName(name);
            product.setQuantity(quantite);
            product.setWeight(weight);
            product.setPrice(price);
        } else {
            product.setQuantity(product.getQuantity() + quantite);

            if (name != null && !name.isBlank()) {
                product.setName(name);
            }
            if (weight != 0 && price.compareTo(BigDecimal.ZERO) != 0) {
                product.setWeight(weight);
                product.setPrice(price);
            }
        }

        if (product.getQuantity() <= 0) {
            product.setStatus("Rupture de stock");
        } else if (product.getQuantity() <= 5) {
            product.setStatus("Stock faible");
        } else if (product.getQuantity() <= 15) {
            product.setStatus("Stock moyen");
        } else {
            product.setStatus("Stock disponible");
        }

        if (name != null) {
            String lowername = name.toLowerCase();
            if (lowername.contains("bœuf")) {
                product.setCategory("Viande rouge");
            } else if (lowername.contains("poulet")) {
                product.setCategory("Volaille");
            } else if (lowername.contains("porc")) {
                product.setCategory("Viande blanche");
            } else {
                product.setCategory("Autre");
            }
        }

        productRepository.save(product);
        return productMapper.toDTO(product);
    }
```

```
    public void updateAllStatuses() {
        List<Product> products = productRepository.findAll();

        for (Product product : products) {
            if (product.getQuantity() <= 0) {
                product.setStatus("Rupture de stock");
            } else if (product.getQuantity() <= 5) {
                product.setStatus("Stock faible");
            } else if (product.getQuantity() <= 15) {
                product.setStatus("Stock moyen");
            } else {
                product.setStatus("Stock disponible");
            }

            productRepository.save(product);
        }
    }

        public ProductDTO removeProduct(String barcode) {
        Product product = productRepository.findByBarcode(barcode)
                .orElseThrow(
                        () -> new ProductNotFoundException("Produit avec code-barres " + barcode +
" introuvable."));

        if (product.getQuantity() <= 0) {
            throw new IllegalStateException("Stock épuisé pour le Produit : " +
product.getName());
        }

        productRepository.save(product);

        return productMapper.toDTO(product);
    }

        public ProductDTO getProductById(int id) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new ProductNotFoundException("Produit avec ID " + id + "
introuvable."));
        return productMapper.toDTO(product);
    }

        public List<ProductDTO> getAllProducts() {
        return productRepository.findAll()
                .stream()
                .map(productMapper::toDTO)
                .collect(Collectors.toList());
    }
    public int getTotalStock() {
        return productRepository.findAll()
                .stream()
                .mapToInt(Product::getQuantity)
                .sum();
    }
```

```java
public class ProductController {

    private final ProductService productService;
    private final ScanService scanService;
    private final UserRepository userRepository;

    @PostMapping("/add")
    public String addOrUpdateProduct(
            @RequestParam String codeBarre,
            @RequestParam(required = false) String nom,
            @RequestParam int quantite,
            @RequestParam int poids,
            @RequestParam BigDecimal prix,
            Model model) {

        ProductDTO product = productService.addProduct(codeBarre, nom,
quantite, poids, prix);
        return "redirect:/products/" + product.getId();

    }

    @GetMapping
    public String listProducts(Model model, Principal principal) {
        User user = userRepository.findByUsername(principal.getName())
                .orElseThrow(() -> new
UserNotFoundException("Utilsateur introuvable"));
        int userId = user.getId();

        productService.updateAllStatuses();
        model.addAttribute("products",
productService.getAllProducts());
        model.addAttribute("stockTotal",
productService.getTotalStock());

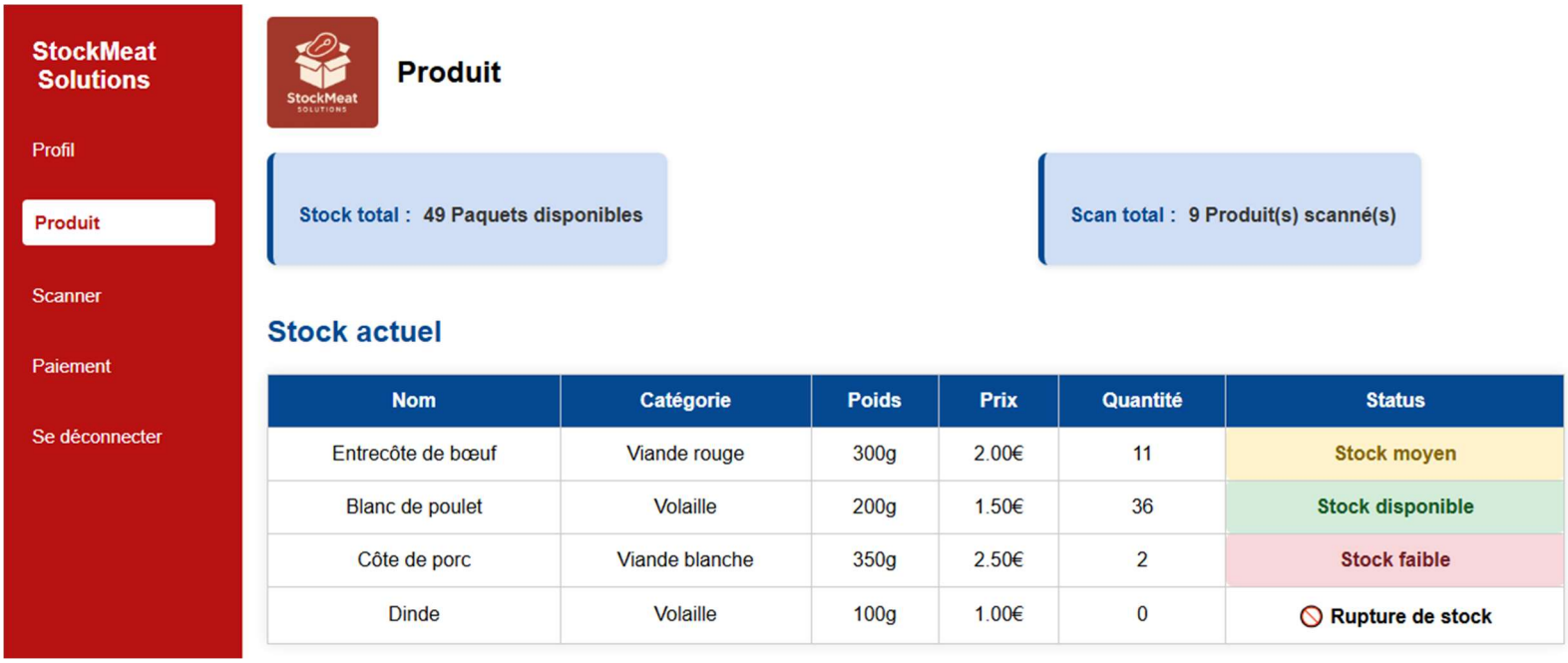
        List<ScanDTO> scans = scanService.getScanDTOsByUser(userId);
        model.addAttribute("scanTotal", scans.size());

        return "products";
    }

    @PostMapping("/remove")
    public String removeProduct(@RequestParam String codeBarre, Model
model) {
        productService.removeProduct(codeBarre);
        return "redirect:/products/";
    }

    @PostMapping("refresh-status")
    public String updateStatus() {
        productService.updateAllStatuses();
        return "redirect:/products";
    }
}
```

Annexe 12 : Interface de gestion des produits



Annexe 13 : Gestion du scan et de la traçabilité (service

```
public class ScanService {

    private final ScanRepository scanRepository;
    private final ScanMapper scanMapper;
    private final UserRepository userRepository;
    private final ProductRepository productRepository;

      public Scan saveScan(int userId, String barcode) {

        User user = userRepository.findById(userId)
                .orElseThrow(() -> new EntityNotFoundException("Utilisateur introuvable"));

        Product product = productRepository.findByBarcode(barcode)
                .orElseThrow(() -> new ProductNotFoundException("Produit introuvable"));

        Scan scan = new Scan();
        scan.setUser(user);
        scan.setProduct(product);
        scan.setDateScan(new Timestamp(System.currentTimeMillis()));

        return scanRepository.save(scan);
    }

        public List<ScanDTO> getScanDTOsByUser(int userId) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new EntityNotFoundException("Utilisateur introuvable"));

        List<Scan> scans = scanRepository.findByUser(user);

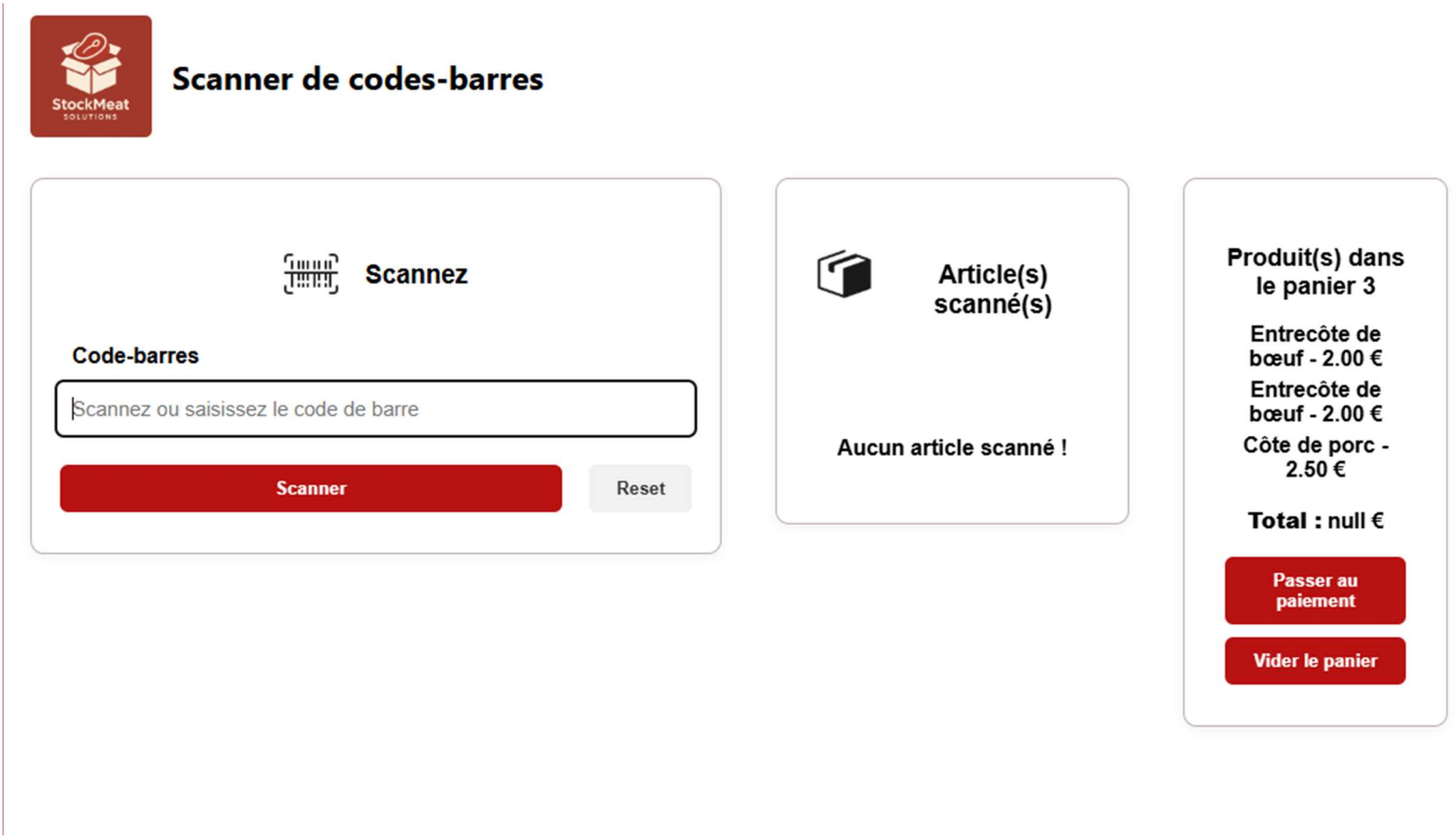
        return scans.stream().map(scanMapper::toDTO).toList();
    }

    public List<ScanDTO> getScanDTOsByProduct(String barcode) {
        Product product = productRepository.findByBarcode(barcode)
                .orElseThrow(() -> new ProductNotFoundException("Produit introuvable"));

        List<Scan> scans = scanRepository.findByProduct(product);
        return scans.stream().map(scanMapper::toDTO).toList();
    }

    public ScanDTO saveScanDTO(int userId, String barcode) {
        Scan scan = saveScan(userId, barcode);
        return scanMapper.toDTO(scan);
    }
```

Annexe 14 : Page de la gestion des Scans



Annexe 15 : Exemple de Code-barres



Code-barres du produit « Dinde »

Annexe 16 : Branchement du Scanner



Annexe 17 : Script AutoHotKey pour l'automatisation du scanner

```
#SingleInstance Force

#Persistent

COM_Port := "COM6"        ;

BaudRate := 9600;

 --- OUVERTURE DU PORT ---

hCom := DllCall("CreateFile", "Str", "\\.\" . COM_Port, "UInt", 0xC0000000, "UInt", 0, "UInt", 0, "UInt", 3, "UInt", 0, "UInt", 0, "Ptr")

if (hCom = 0 || hCom = -1) {

   MsgBox, ❌ Échec ouverture %COM_Port%`nVérifie que le scanner est branché et libre

   ExitApp };

 --- CONFIGURATION DU PORT ---

VarSetCapacity(dcb, 80, 0)

NumPut(80, dcb, 0, "UInt")

DllCall("GetCommState", "Ptr", hCom, "Ptr", &dcb)

NumPut(BaudRate, dcb, 4, "UInt")    ; BaudRate

NumPut(1, dcb, 8, "UChar")        ; fBinary

NumPut(0, dcb, 9, "UChar")        ; Parity

NumPut(8, dcb, 23, "UChar")        ; ByteSize

NumPut(0, dcb, 24, "UChar")        ; StopBits

NumPut(0, dcb, 25, "UChar")        ; FlowControl

DllCall("SetCommState", "Ptr", hCom, "Ptr", &dcb)

; --- TIMEOUTS ---

VarSetCapacity(cto, 20, 0)

NumPut(50, cto, 0, "UInt")        ; ReadIntervalTimeout

NumPut(0, cto, 4, "UInt")         ; ReadTotalTimeoutMultiplier

NumPut(50, cto, 8, "UInt")        ; ReadTotalTimeoutConstant

NumPut(0, cto, 12, "UInt")        ; WriteTotalTimeoutMultiplier

NumPut(50, cto, 16, "UInt")       ; WriteTotalTimeoutConstant

DllCall("SetCommTimeouts", "Ptr", hCom, "Ptr", &cto)
```

Annexe 18 : Notice d'utilisation du Scanner

Datalogic Scanning, Inc.
959 Terry Street
Eugene, Oregon 97402
USA
Telephone: (541) 683-5700
Fax: (541) 345-7140

Un Travail non publié : Tous droits réservés. Aucune partie du contenu de cette documentation ou des procédures décrites dans celle-ci ne peut être reproduite ou diffusée sous aucune forme ni par aucune méthode sans l'accord écrit de la part de Datalogic Scanning, Inc. ou celui de ses filiales ou de ses associées ("Datalogic" ou "Datalogic Scanning"). Par cette présente, aux propriétaires des produits de Datalogic est garantie une licence révocable et non exclusive de reproduction et diffusion interne de cette documentation dans un but purement commercial. Les acheteurs ne doivent ni ôter ni altérer les avis du propriétaire, y compris les avis de droit d'auteur contenus dans cette documentation et ils doivent s'assurer que tous les avis apparaissent sur toutes les reproductions de cette documentation. En cas de publication de révisions futures de ce manuel, vous pouvez les recevoir en contactant notre représentant Datalogic Les versions électroniques peuvent être téléchargées à partir du site Internet de Datalogic (www.scanning.datalogic.com) ou elles peuvent être fournies sur des supports appropriés. Si vous visitez notre site Internet et que vous désirez faire des commentaires ou exprimer des suggestions à propos de cette publication ou d'autres publications de Datalogic, nous vous prions de nous les laisser sur la page "Contact Datalogic".

DENEGATION DE RESPONSABILITE

Datalogic a pris des mesures opportunes pour fournir des informations complètes et soignées dans ce manuel. Cependant Datalogic se réserve le droit de changer n'importe quel détail à n'importe quel moment sans avis préalable. Datalogic est une marque enregistrée de Datalogic S.p.A. dans plusieurs pays, le logo de Datalogic est une marque de Datalogic S.p.A., l'une et l'autre sont licenciées à Datalogic Scanning, Inc.
Toutes les autres marques déposées et les appellations commerciales qui paraissent dans ce manuel appartiennent à leurs propriétaires respectifs.

SERVICES ET AIDE TECHNIQUE

Datalogic fournit différents services et une aide technique en ligne à l'adresse www.scanning.datalogic.com. Cliquez sur les liens indiqués pour avoir les informations désirées:

- **PRODUITS**
  Naviguez parmi les liens pour arriver à la page de votre produit d'où vous pouvez télécharger des **Manuels** ainsi que des logiciels & programmes utilitaires (**Software & Utilities**) y compris:
  - **Datalogic Aladdin™** un programme utilitaire multi-plateforme qui permet la configuration de l'appareil en utilisant l'ordinateur, la configuration à travers l'interface RS-232 ainsi que l'impression de codes à barres de configuration.
- **SERVICE & SUPPORT**
  - **Support Technique** – Documentation du produit et guides de programmation. Support technique dans le monde
  - **Programmes de service** – Termes de garantie et d'entretien
  - **Service de reparation** – Réparations à prix fixe et réparations RMA (Return Material Authorization).
  - **Télécharger** – Manuels & Documentation, Data Sheets, Catalogues, etc.
- **CONTACTEZ-NOUS**
  Formulaire pour la requête d'information & réseau de ventes et services.

Datalogic Scanning, Inc.
959 Terry Street
Eugene, OR 97402
USA

CE 07

dichiara che
declares that the
déclare que le
bescheinigt, daß das Gerät
declara que el

Touch 65
Touch 90

e tutti i suoi modelli
and all its models
et tous ses modèles
und seine Modelle
y todos sus modelos

sono conformi alle Direttive del Consiglio Europeo sottoelencate:
are in conformity with the requirements of the European Council Directives listed below:
sont conformes aux spécifications des Directives de l'Union Européenne ci-dessous:
den nachstehenden angeführten Direktiven des Europäischen Rats:
cumple con los requisitos de las Directivas del Consejo Europeo, según la lista siguiente:

89/336/EEC EMC Directive e **92/31/EEC, 93/68/EEC** emendamenti successivi
and further amendments
et ses successifs amendements
und späteren Abänderungen
y sucesivas enmiendas

Basate sulle legislazioni degli Stati membri in relazione alla compatibilità elettromagnetica ed alla sicurezza dei prodotti.
On the approximation of the laws of Member States relating to electromagnetic compatibility and product safety.
Basée sur la législation des Etats membres relative à la compatibilité électromagnétique et à la sécurité des produits.
Über die Annäherung der Gesetze der Mitgliedsstaaten in bezug auf elektromagnetische Verträglichkeit und Produktsicherheit entsprechen.
Basado en la aproximación de las leyes de los Países Miembros respecto a la compatibilidad electromagnética y las Medidas de seguridad relativas al producto.

Questa dichiarazione è basata sulla conformità dei prodotti alle norme seguenti:
This declaration is based upon compliance of the products to the following standards:
Cette déclaration repose sur la conformité des produits aux normes suivantes:
Diese Erklärung basiert darauf, daß das Produkt den folgenden Normen entspricht:
Esta declaración se basa en el cumplimiento de los productos con la siguientes normas:

EN 55022 (CLASS B ITE), AUGUST 1994. AMENDMENT A1 (CLASS B ITE), OCTOBER 2000: LIMITS AND METHODS OF MEASUREMENTS OF RADIO DISTURBANCE OF INFORMATION TECHNOLOGY EQUIPMENT

EN 55024, SEPTEMBER 1998: INFORMATION TECHNOLOGY EQUIPMENT. IMMUNITY CHARACTERISTICS. LIMITS AND METHODS OF MEASUREMENTS

March 1st, 2007

DATALOGIC

Touch™ 65/90 Light

CONTACT READERS



GUIDE RAPIDE

©2012 Datalogic Scanning, Inc. 820048115 (Rev. B) March 2012

China RoHS

DATALOGIC™

PART	Lead (Pb)	Mercury (Hg)	Cadmium (Cd)	Hexavalent Chromium (Cr(VI))	Polybrominated biphenyls (PBB)	Polybrominated diphenyl ethers (PBDE)
部件名称	铅	汞	镉	六价铬	多溴联苯	多溴二苯醚
有害有毒物质或元素						
Printed Circuit Board Assembly 电路板组件	X	O	O	O	O	O
Assy. Optics Block 光学组件	X	O	O	O	O	O
Assy. Module 模块组件	X	O	O	O	O	O

O: 代表此部件所有均质材料中所含有毒有害物质均低于SJ/T 11363-2006 规定的限量。
O: 表示该有毒有害物质在该部件所有均质材料中的含量均在SJ/T 11363-2006 规定的限量要求以下。
X: 代表此部件所用的均质材料中，至少有一类材料其所含有毒有害物质高于SJ/T 11363-2006 规定的限量要求。
X: 表示该有毒有害物质至少在该部件的某一均质材料中的含量超出SJ/T 11363-2006 规定的限量要求。

CONNEXIONS

USB



INTERCLAVIER



RS-232



NOTE *Avant de connecter le lecteur, vérifiez que votre câble de connexion correspond à l'interface du modèle Touch™ Light (voir l'étiquette du produit). Ne connectez pas des câbles d'interface qui ne sont pas compatibles avec votre modèle de Touch™ Light.*

DECONNECTER LE CABLE



UTILISATION DU LECTEUR TOUCH™ LIGHT





Avec les lecteurs Touch™ Light, la lecture des codes s'effectue automatiquement et **par contact**. Tous les modèles sont doués d'un poussoir qui doit par défaut être appuyé pour lire le code. Après avoir configuré l'interface de lecture, on peut sélectionner le paramètre software trigger (disponible dans ce manuel) pour utiliser le lecteur comme un modèle sans poussoir.

Le code est lu par le faisceau sortant de la fenêtre de lecture de l'appareil. Ce faisceau doit traverser le code à barres à lire de part en part.
Pour des prestations optimales tenez le manche de la douchette parallèlement au code à lire (voir la figure ci-dessus).

CONFORMITE

CLASSE DE LA LED

LED de Classe 1.

EN60825-1:2001

BREVETS

Ce produit est sous licence pour le suivant brevet des Etats Unis:

6,158,661

Ce produit est couvert par un ou plusieurs brevets parmi les suivants:

Brevets des Etats Unis 5,917,176; 5,992,740; 6,010,073; 6,305,606 B1; 6,517,003 B2; and 6,712,271 B2

Brevets européens 789,315 B1, 822,508 B1; 829,814 B1; 851,378 B1; 895,175 B1; et 1,205,871 B1

D'autres brevets en cours de validation.

CONFORMITE DEEE

Waste Electrical and Electronic Equipment (WEEE) Statement

English
For information about the disposal of Waste Electrical and Electronic Equipment (WEEE), please refer to the website at www.scanning.datalogic.com.

Italian
Per informazioni sullo smaltimento delle apparecchiature elettriche ed elettroniche consultare il sito Web www.scanning.datalogic.com.

French
Pour toute information relative à l'élimination des déchets électroniques (WEEE), veuillez consulter le site Internet www.scanning.datalogic.com.

German
Informationen zur Entsorgung von Elektro- und Elektronik- Altgeräten (WEEE) erhalten Sie auf der Webseite www.scanning.datalogic.com.

Spanish
Si desea información acerca de los procedimientos para el desecho de los residuos del equipo eléctrico y electrónico (WEEE), visite la página Web www.scanning.datalogic.com.

Portuguese
Para informações sobre a disposição de Sucatagem de Equipamentos Eléctricos e Eletrônicos (WEEE - Waste Electrical and Electronic Equipment), consultar o site web www.scanning.datalogic.com.

Chinese
有关处理废弃电气电子设备 (WEEE) 的信息， 请参考Datalogic公司的网站 http://www.scanning.datalogic.com/。

Japanese
廃電気電子機器(W E E E)の処理についての 関連事項はDatalogicのサイト www.scanning.datalogic.com, をご参照下さい。

CONFIGURATION PAR DEFAUT

CONFIGURATION INTERFACE USB Standard

CLAVIER USB: clavier américain, délai entre les caractères et délai entre les codes désactivés, gestion du caractère de contrôle = Ctrl+Shift+touche.

FORMAT DE DONNEES: code d'identification désactivé, ajustement de champ désactivé, transmission de la longueur du code désactivé, "terminator" = ENTER, substitution de caractère désactivée.

CONFIGURATION INTERFACE RS-232 Standard

9600 baud, parité désactivée, 8 bit de données, 1 bit de stop, "handshaking" désactivé, délai entre le caractères désactivé, "rxtimeout" 5 sec., ACK/NACK désactivé, FIFO activé, contrôle sériel de la gâchette désactivé.

FORMAT DE DONNEES: code d'identification désactivé, ajustement de champ désactivé, transmission de la longueur du code désactivé, aucun "header", "terminator" = CR-LF, substitution de caractère désactivée.

CONFIGURATION INTERFACE INTERCLAVIER

Clavier américain, "caps lock" inactif, caps lock auto-recognition activé; pavé numérique inchangé, délai entre les caractères et délai entre les codes désactivés, gestion du caractère de contrôle = Ctrl+Shift+touche.

FORMAT DE DONNEES: code d'identification désactivé, ajustement de champ désactivé, transmission de la longueur du code désactivé, aucun "header", "terminator" = ENTER, substitution de caractère désactivé.

CONFIGURATION INTERFACE CRAYON

Mode "interpret" activé, conversion vers le code 39 désactivé, niveau de sortie normal, niveau de "idle" normal, sortie minimum d'impulsions 600 µs, "overflow" moyen, délai "inter-block" désactivé.

SELECTION DU CODE

Codes activés

- EAN 8/EAN 13 / UPC A/UPC E sans ADD ON
  Transmission du check digit, pas de conversion
- Interleaved 2/5
  contrôle et transmission du check digit, longueur variable de 4 à 99 caractères
- Standard Code 39
  contrôle du check digit désactivé, longueur variable de 1 à 99 caractères
- Code 128
  longueur variable de 1 à 99 caractères

Codes désactivés

EAN 128, ISBT128, Code 93, Codabar, codes pharmaceutiques, MSI, Plessey, Telepen, Delta IBM, Code 11, Code 16K, Code 49, RSS Codes

CARACTERISTIQUES TECHNIQUES

Caractéristiques électriques		
Alimentation RS232 interface	5 Vdc ± 5%	
Consommation: Maximum Fonctionnement Mode veille & Mode USB	150 mA @ 5 Vdc 100 mA @ 5 Vdc < 500 µA @ 5 Vdc	
Vitesse de scansion max.	256 scans/sec	
Indicateurs de lecture	LED, Beeper	
Caractéristiques laser optiques		
	Touch™ 65 Light	**Touch™ 90 Light**
Capteur	CCD solid state (2088 pixels)	
Champ de lecture	63 mm typique	80 mm typique
Résolution max.	0.10 mm (4 mils)	0.13 mm (5 mils)
PCS	min. 25% (Datalogic Test Chart)	
Conditions ambiantes		
Temp. de fonctionnement	0° to +55 °C	
Temp. de stockage	-20° to +70 °C	
Humidité	90% sans condensation	
Résistance aux chutes	1.2 m	
Protection ESD	16 KV	
Classe de Protection	IP30	
Caractéristiques mécaniques		
Poids (sans cable)	160 g environ	195 g environ
Longueur du cable	2 m	

CONFIGURATION PAR DÉFAUT

CONFIGURATION PAR DÉFAUT

Pour changer les paramètres voir le manuel "HHD II Software Configuration Manual", ou bien le programme de configuration DL Sm@rtSet™. Le manuel et le programme peuvent être déchargés de notre site internet.

SELECTION DE L'INTERFACE

Lire les codes de configuration de l'interface selon votre application.

SELECTION DE L'INTERFACE USB

USB-KBD

USB-KBD - ALT-mode

USB-COM *

USB-IBM-Hand Held

* Lors de la configuration de l'interface USB-COM, les fichiers et drivers correspondants doivent être installés à partir du logiciel d'installation pour dispositifs USB qui peut être téléchargé sur la page web: http://www.scanning.datalogic.com.

CONFIGURATION DU LECTEUR EN RS232

RS-232 Standard

Nixdorf Mode A

Fujitsu

ICL Mode

CONFIGURATION DU LECTEUR EN INTERCLAVIER

Wedge IBM AT or PS/2 PCs

PC Notebook

PC Notebook - ALT mode

IBM AT - ALT mode

D'autres interfaces sont également supportées et peuvent être sélectionnées dans le manuel "HHD II Software Configuration Manual", qui peut être téléchargé sur la page web: http://www.scanning.datalogic.com. Les interfaces supportées sont:

USB: USB-IBM-Table Top; USB-KBD-APPLE

WEDGE: IBM XT; IBM SURE1; IBM Terminal 3153; IBM Terminals 31xx, 32xx, 34xx, 37xx; Wyse Terminals ANSI – PC –ASCII – VT220 style Keyboards; Digital Terminal VT2xx/VT3xx/VT4xx; APPLE ADB Bus

NATIONALITE DU CLAVIER

Après avoir configuré l'interface USB-KBD ou interclavier, lire l'un des codes suivants pour sélectionner la nationalité du clavier.

Belge

Deutsch

English

Español

Français

Italiano

Japanese

Svenskt

USA (Default)

"TERMINATORS" DU FORMAT DE DONNÉES

Pour vous faciliter, les "terminators" les plus courants sont disponibles ci-dessous. D'autres "header/terminators" peuvent être sélectionnés dans le manuel "HHD II Software Configuration Manual", voir les paramètres Data Format et Advanced Data Format.

CR-LF

Enter

Tab

None

TEST DE FONCTIONNEMENT

Lire les codes tests.

Code 128

t e s t

MODE DU TRIGGER

Après avoir configuré l'interface de lecture, on peut sélectionner le paramètre software trigger pour utiliser le lecteur comme un modèle sans poussoir.

Software Trigger

• Hardware Trigger

GARANTIE D'USINE DE DATALOGIC SCANNING

Couverture de la garantie

Datalogic garantit ce produit de tout défaut de fabrication ou des matériaux pendant 5 ans à compter de la date de livraison, à condition que le produit soit utilisé correctement.

Le matériel de Datalogic Scanning (ci-après dénommée « Datalogic ») est assorti d'une garantie contre les défauts de fabrication et de qualité dans les conditions normales et habituelles d'utilisation. Au terme de cette garantie, la responsabilité de Datalogic se limite à la mise en œuvre des efforts et à la fourniture des éléments nécessaires à la résolution de tout défaut couvert par la garantie et à la remise en état de fonctionnement normal du produit. Toute réparation ou tout remplacement de produit durant la période de garantie ne prolonge en aucun cas la durée de garantie initiale. Les produits sont commercialisés sur la base des spécifications applicables au moment de leur fabrication. Il n'appartient en aucun cas à Datalogic de modifier ou de mettre à jour les produits qui ont été vendus.

Si Datalogic établit qu'un produit présente un quelconque défaut de fabrication ou de qualité, elle s'engage à exclusivement réparer ou remplacer le produit concerné sans frais supplémentaires, ni pour les pièces, ni pour la main d'œuvre, ou à créditer ou rembourser les produits défectueux qui lui auront été dûment retournés. Dans le cadre des réparations, Datalogic se réserve le droit d'utiliser des pièces, composants ou sous-composants neufs ou d'occasion ou des produits qui ont été testés et qui répondent aux spécifications requises comparables à celles des produits ou du matériel neufs équivalents. Le Client autorise Datalogic à éliminer toutes les pièces enlevées dans le cadre de la réparation des produits. La période de garantie s'étendra de la date d'expédition par Datalogic pour toute la durée annoncée par Datalogic au moment de l'achat du produit (Durée de garantie). Datalogic garantit le matériel réparé contre les vices de fabrication et le matériel des pièces réparées pour une durée de 90 jours à partir de la date d'expédition par Datalogic du produit réparé ou jusqu'à la fin de la durée de garantie initiale, en fonction de la durée. Datalogic ne garantit pas et décline toute responsabilité en matière de maintenance, de dommage ou de perte des configurations, des données et des applications des pièces réparées, et se réserve le droit, à sa seule discrétion, de restaurer la configuration d'usine ou d'installer toute mise à jour de tout logiciel ou de micrologiciel disponible au moment de la réparation (autre que le logiciel ou micrologiciel installé lors de la fabrication du produit). Le Client assume l'entière responsabilité de la réalisation régulière de copie de sauvegarde de ses logiciels et données.

Procédure de réclamation

Afin de faire appel à la garantie d'usine, le Client est tenu de notifier le défaut supposé avant l'expiration de la garantie applicable et d'obtenir de sa part un numéro d'autorisation de retour (RMA) afin de retourner le produit à un centre de services désigné par Datalogic. Si Datalogic établit que la plainte du Client est recevable, elle procédera à la réparation ou au remplacement à ses frais. Le Client assume la responsabilité du conditionnement et de l'expédition du produit au centre de services désigné par Datalogic, sans la moindre intervention financière de la part de cette dernière dans les frais d'expédition. Datalogic assume les frais d'expédition au Client pour autant qu'il s'agisse d'une adresse située dans le pays où se trouve le centre de services de Datalogic. Le Client assume tous les frais d'expédition, droits, taxes et tout autre supplément liés à l'expédition dans un autre pays. Le non-respect de cette procédure de RMA entraînera des frais de procédure. En outre, le Client assumera les frais d'expédition des produits que Datalogic aura jugé, à sa seule discrétion, non défectueux ou non couverts par la garantie.

Exclusions

La garantie d'usine de Datalogic ne s'applique pas aux éléments suivants :

(i) tout produit endommagé, modifié, altéré, réparé ou mis à jour par un tiers autre qu'un membre du personnel de Datalogic ou l'un de ses représentants agréés ;

(ii) tout défaut, panne ou dégât présumé que Datalogic impute à de mauvaises manipulations, à une mauvaise utilisation, à des abus, utilisations abusives, usure, négligence, stockage inadéquat ou utilisation de pièces ou accessoires non agréés ou fournis par Datalogic ;

(iii) tout défaut ou dégât présumé causé par l'utilisation du produit avec tout autre instrument, équipement ou appareil ;

(iv) tout défaut ou dégât présumé causé par une mauvaise gestion de la maintenance, en ce compris mais pas exclusivement le nettoyage de la fenêtre supérieure conformément au manuel d'utilisation du produit ;

(v) tout défaut ou dégât provoqué par une catastrophe naturelle ou technologique par exemple mais pas exclusivement le feu, les dégâts des eaux, les inondations ou autre catastrophe naturelle, le vandalisme ou tout abus susceptible de provoquer des dégâts aux composants internes ou externes ou la destruction de la pièce entière, des consommables ;

(vi) tout dégât ou dysfonctionnement provoqué par l'absence d'opération d'entretien telle que mises à jour des logiciels ou micrologiciels, reconfigurations des logiciels ou du matériel, etc. ;

(vii) le remplacement de la fenêtre/cartouche supérieur(e) en raison de griffes, taches ou autre dégradation et/ou

(viii) tout consommable ou produit assimilé (par ex. câbles, blocs d'alimentation, batteries, claviers, écrans tactiles, interrupteurs, etc.).

Cession

Le Client n'a en aucun cas le droit de céder ou de transférer les droits ou obligations qui découlent de la présente garantie, exception faite d'un acheteur ou d'un cessionnaire du produit. Aucune tentative de cession ou de transfert contraire à cette disposition ne sera valide ou contraignante dans le chef de Datalogic.

CETTE GARANTIE LIMITÉE DE DATALOGIC REMPLACE TOUTE AUTRE GARANTIE, EXPRESSE OU TACITE, ÉCRITE OU ORALE, STATUTAIRE OU AUTRE, EN CE COMPRIS MAIS PAS EXCLUSIVEMENT, TOUTE GARANTIE TACITE COMMERCIALE, CONVENANT À DES FINS PARTICULIÈRES OU NON-CONTREFAÇON. DATALOGIC NE POURRA EN AUCUN CAS ÊTRE TENUE RESPONSABLE DE TOUT DÉGÂT DÉCOULANT DE LA NÉGLIGENCE DU CLIENT EN RAISON D'UN QUELCONQUE RETARD DANS LE REMPLACEMENT OU LA RÉPARATION DES PRODUITS TEL QUE PRÉCISÉ CI-DESSUS. LES SOLUTIONS EXPOSÉES DANS LE CADRE DE LA PRÉSENTE GARANTIE SONT LES SEULES ET UNIQUES SOLUTIONS POUR TOUTE PLAINTE. DATALOGIC NE POURRA EN AUCUN CAS ÊTRE TENUE RESPONSABLE PAR LE CLIENT OU UN TIERS DE TOUTE PERTE OU AUTRE DÉGÂT DIRECT, SPÉCIAL OU CONTINGENT, ACCIDENTEL OU CONSÉCUTIF INDÉPENDAMMENT DU FAIT QUE DATALOGIC AIT PU OU NON PRÉVOIR DE TELS DÉGÂTS.

Risque de perte

Le Client assume l'entière responsabilité du risque de perte ou de détérioration du produit lors de l'expédition au centre de service de Datalogic. Datalogic assume l'entière responsabilité du risque de perte ou de détérioration des produits en sa possession. En l'absence d'instructions écrites spécifiques de renvoi au Client, Datalogic choisit le transporteur mais décline toute responsabilité en matière d'expédition.

Annexe 19 : Interface de la gestion de Paiement



Annexe 20 : Exemple de Facture générée (PDF)



Facture - StockMeat Solutions

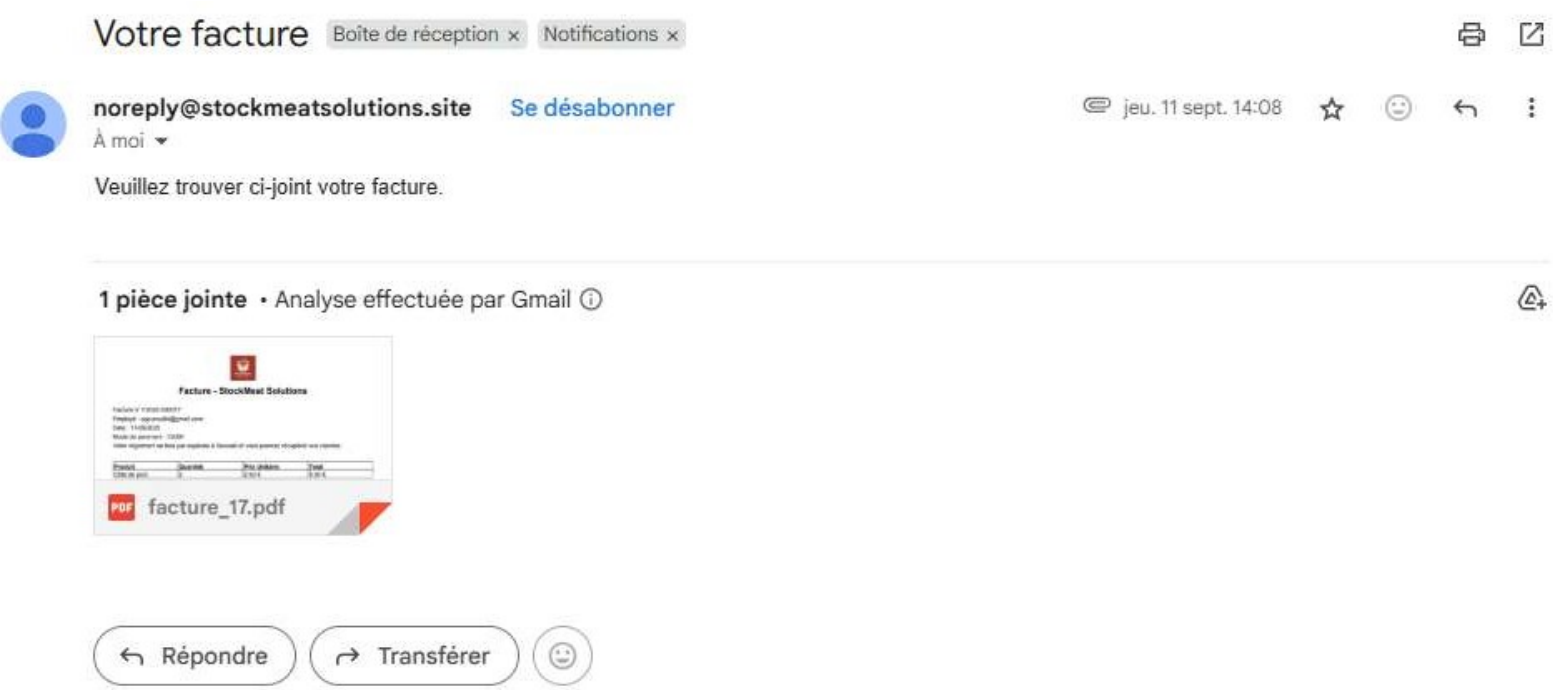
Facture n° F2025-000017
Employé : agourou84@gmail.com
Date : 11/09/2025
Mode de paiement : CASH
Votre règlement se fera par espèces à l'accueil et vous pourrez récupérer vos viandes.

Produit	Quantité	Prix Unitaire	Total
Côte de porc	2	2.50 €	5.00 €

Montant total : 5.00 €

StockMeat Solutions • Ne pas répondre à ce mail

Annexe 21 : Mail reçue avec Facture en pièce jointe



Annexe 22 : Fichier .env

```
SPRING_MAIL_PASSWORD=**********

SPRING_DATASOURCE_PASSWORD=********

SSL_KEYSTORE_PASSWORD=********************

SPRING_PROFILES_ACTIVE=dev

SPRING_MAIL_USERNAME=9583ad001@smtp-brevo.com
```

Annexe 23 : Fichier docker-compose.yml

```yaml
volumes:
  mysql-data:
networks:
  spring-boot-mysql-network:
    name: spring-boot-mysql-network
▷Run All Services
services:
  ▷Run Service
  mysqldb:
    image: mysql:lts

    ports:
      - "3306:3306"
    environment:
      - MYSQL_DATABASE=project_apply
      - MYSQL_ROOT_PASSWORD=rootroot
    networks:
      - spring-boot-mysql-network
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -prootroot"]
      interval: 10s
      timeout: 60s
      retries: 10
```

```yaml
stockmeatsolutions:
  build: .
  ports:
    - "8080:8080"
  depends_on:
    mysqldb:
      condition: service_healthy
  env_file:
    - .env
  environment:
    - SPRING_PROFILES_ACTIVE=dev
    - SPRING_APPLICATION_NAME=Project-Apply

    - SPRING_DATASOURCE_URL=jdbc:mysql://mysqldb:3306/project_apply?serverTimezone=UTC
    - SPRING_DATASOURCE_USERNAME=root
    - SPRING_DATASOURCE_DRIVERCLASSNAME=com.mysql.cj.jdbc.Driver

    - SPRING_JPA_HIBERNATE_DDL_AUTO=update
    - SPRING_JPA_SHOW_SQL=true
    - SPRING_JPA_PROPERTIES_HIBERNATE_FORMAT_SQL=true
    - SPRING_JPA_PROPERTIES_HIBERNATE_DIALECT=org.hibernate.dialect.MySQLDialect

    - SPRING_MAIL_HOST=smtp-relay.brevo.com
    - SPRING_MAIL_PORT=587
    - SPRING_MAIL_USERNAME=9583ad001@smtp-brevo.com
    - SPRING_MAIL_PASSWORD=${SPRING_MAIL_PASSWORD}
    - SPRING_MAIL_PROPERTIES_MAIL_SMTP_AUTH=true
    - SPRING_MAIL_PROPERTIES_MAIL_SMTP_STARTTLS_ENABLE=true

  networks:
    - spring-boot-mysql-network
```

Annexe 24 : Commande Docker

docker compose up --build -d → **Construit** les images et **lance** les conteneurs **en arrière-plan**

docker compose down → **Arrête** et **supprime** les conteneurs, réseaux, volumes

docker ps → **Liste** les conteneurs **actifs**

docker compose build --no-cache → **Reconstruit** les images **sans utiliser le cache**

Annexe 25 : Exemples de tests unitaires et d'intégration (ProductServiceTest & ProductRepositoryIT)

```java
public class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;
    @Mock
    private ProductMapper productMapper;
    @InjectMocks
    private ProductService productService;

    private Product product;

    @BeforeEach
    @SuppressWarnings("unused")
    void setUp() {
        product = new Product();
        product.setId(1);
        product.setName("Bœuf");
        product.setBarcode("123456789");
        product.setQuantity(20);
        product.setWeight(100);
        product.setPrice(new BigDecimal("5.00"));
    }

    @Test
    void testAddProduct_existingProduct_updateFields() {
        Product existing = new Product();
        existing.setBarcode("123456789");
        existing.setQuantity(5);
        existing.setName("Ancien nom");
        existing.setPrice(new BigDecimal("2.00"));
        existing.setWeight(50);

        when(productRepository.findByBarcode("123456789")).thenReturn(Optional.of(existing));
        when(productRepository.save(any())).thenReturn(existing);
        when(productMapper.toDTO(any())).thenReturn(new ProductDTO());

        ProductDTO result = productService.addProduct("123456789", "Bœuf", 10, 100, new BigDecimal("5.00"));
        assertNotNull(result);
        verify(productRepository).save(existing);
    }

    @Test
    void testAddProduct_categoryAutre() {
        when(productRepository.findByBarcode("000")).thenReturn(Optional.empty());
        when(productRepository.save(any())).thenReturn(product);
        when(productMapper.toDTO(any())).thenReturn(new ProductDTO());

        ProductDTO result = productService.addProduct("000", "Mystère", 10, 100, new BigDecimal("1.00"));
        assertNotNull(result);
        verify(productRepository).save(any());
    }
```

```java
public class ProductRepositoryIT {

    @Autowired
    private ProductRepository productRepository;

    @Test
    void testSaveAndFindProduct() {
        Product product = new Product();

        product.setName(name:"Steak");
        product.setBarcode(barcode:"14785296325887");
        product.setQuantity(quantity:4);
        product.setWeight(weight:300);
        product.setPrice(new BigDecimal(val:"2.50"));

        Product saved = productRepository.save(product);
        assertNotNull(saved.getId());

        List<Product> all = productRepository.findAll();
        assertFalse(all.isEmpty());
        assertEquals(expected:"Steak", all.get(index:0).getName());
    }
}
```

Annexe 26 : Rapports JaCoCo et Surefire

Project-Apply

Project-Apply

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.projetApply.Project_Apply.service		89 %		65 %
com.projetApply.Project_Apply.exception		42 %		n/a
com.projetApply.Project_Apply.controller		95 %		90 %
com.projetApply.Project_Apply.configuration		97 %		n/a
com.projetApply.Project_Apply.mapper		98 %		80 %
com.projetApply.Project_Apply.model		100 %		n/a
com.projetApply.Project_Apply		100 %		n/a
Total	233 of 2 583	90 %	28 of 109	74 %

Project-Apply > com.projetApply.Project_Apply.service

com.projetApply.Project_Apply.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
ProductService		81 %		76 %
PaymentService		83 %		56 %
MailServiceImplements		84 %		n/a
InvoiceService		95 %		66 %
UserService		91 %		37 %
ScanService		100 %		n/a
Total	133 of 1 235	89 %	23 of 67	65 %

Dernière publication: 2025-09-24 | Version: 0.0.1-SNAPSHOT

Produit par Maven

Surefire Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Success Rate	Time
93	0	0	0	100%	105,335

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[Summary] [Package List] [Test Cases]

Package	Tests	Errors	Failures	Skipped	Success Rate	Time
com.projetApply.Project_Apply.unitaire.controller	42	0	0	0	100%	28,548
com.projetApply.Project_Apply.unitaire.mapper	12	0	0	0	100%	0,313
com.projetApply.Project_Apply	1	0	0	0	100%	7,363
com.projetApply.Project_Apply.unitaire.configuration	5	0	0	0	100%	57,083
com.projetApply.Project_Apply.unitaire.service	33	0	0	0	100%	12,028